

On State-Restoration Knowledge Soundness from Special Soundness

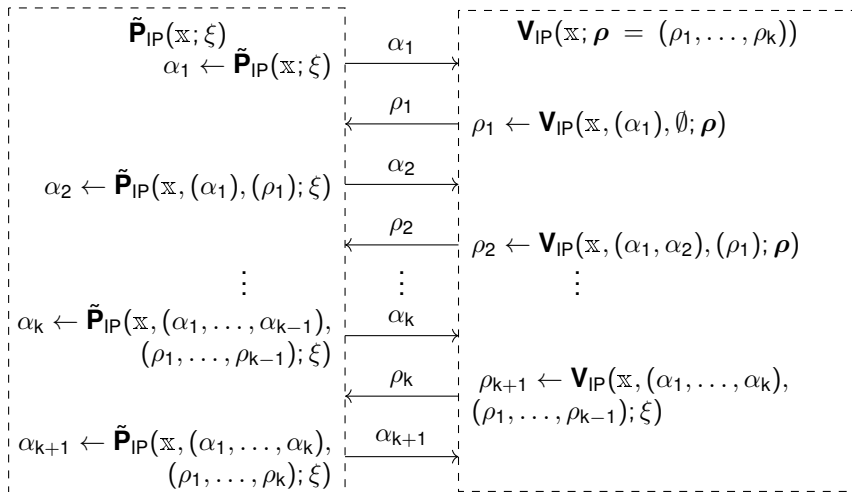
Mathias Marty

EPFL – COMPSEC

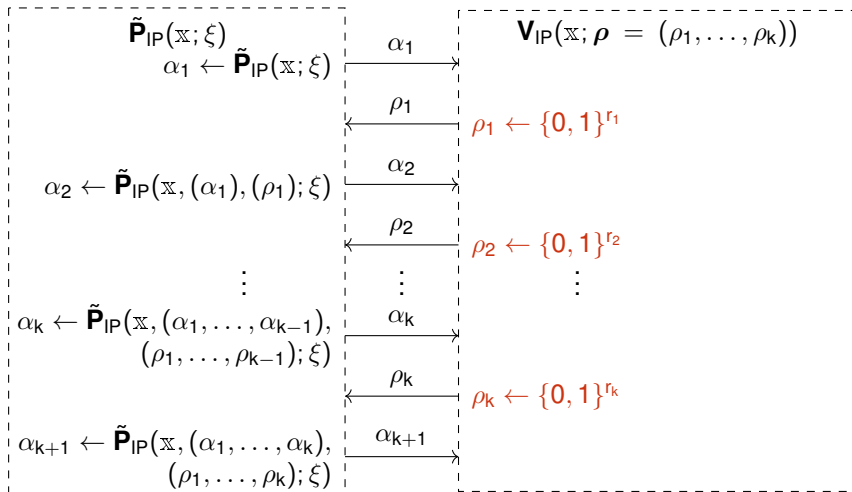
July 2024

Preliminaries

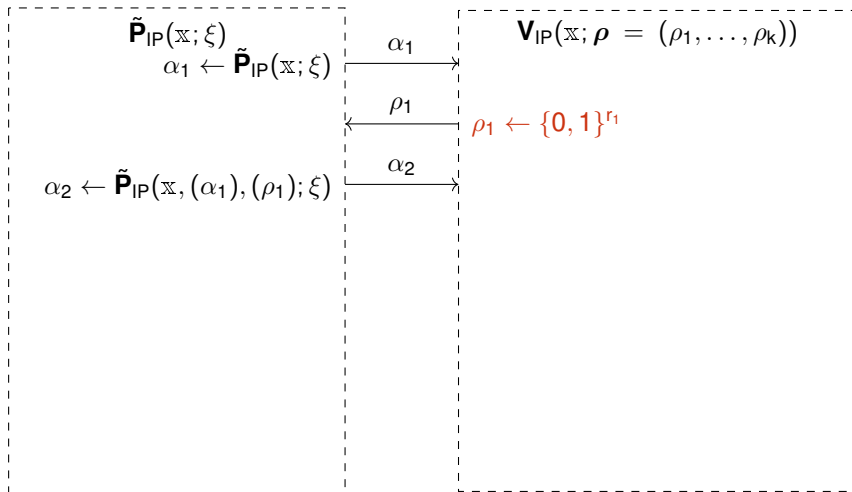
Interactive proofs



Interactive proofs



Interactive proofs



Completeness and soundness

Let IP be an interactive proof (IP) for the relation R .

Definition (Completeness)

If $x \in L(R)$, then the honest prover always convinces the honest verifier that $x \in L(R)$.

Completeness and soundness

Let IP be an IP for the relation R .

Definition (Completeness)

If $x \in L(R)$, then the honest prover always convinces the honest verifier that $x \in L(R)$.

Definition (Soundness)

If $x \notin L(R)$, then any malicious prover cannot convince the honest verifier, except with low probability, that $x \in L(R)$.

Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{\mathbf{P}}_{\text{IP}}$ convinces us, the instance $\mathbf{x}$ is in the language $L(R)$;

Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{\mathbf{P}}_{\mathbf{IP}}$ convinces us, the instance $\mathbf{x}$ is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness $\mathbf{w}$ for $\mathbf{x}$;

Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{\mathbf{P}}_{\text{IP}}$ convinces us, the instance $\mathbf{x}$ is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness $\mathbf{w}$ for $\mathbf{x}$;
- 3 This motivates the security notion of *knowledge soundness*!

Knowledge soundness

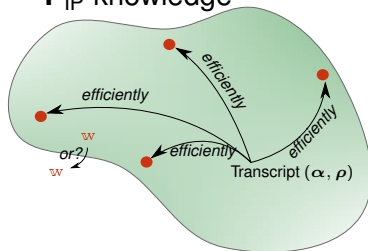
- 1 Soundness ensures that if any (malicious) prover $\tilde{\mathbf{P}}_{\text{IP}}$ convinces us, the instance $\mathbf{x}$ is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness $\mathbf{w}$ for $\mathbf{x}$;
- 3 This motivates the security notion of *knowledge soundness*!
- 4 But how can we capture the *knowledge* of $\tilde{\mathbf{P}}_{\text{IP}}$?

Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{P}_{IP}$ convinces us, the instance x is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness w for x ;
- 3 This motivates the security notion of *knowledge soundness*!
- 4 But how can we capture the *knowledge* of $\tilde{P}_{IP}$?

What it knows for sure:

$\tilde{P}_{IP}$ knowledge

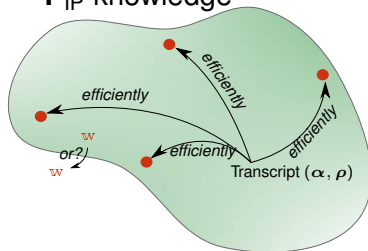


Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{P}_{IP}$ convinces us, the instance x is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness w for x ;
- 3 This motivates the security notion of *knowledge soundness*!
- 4 But how can we capture the *knowledge* of $\tilde{P}_{IP}$?

What it knows for sure:

$\tilde{P}_{IP}$ knowledge

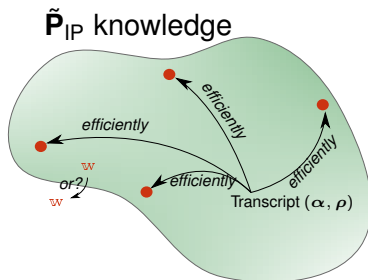


- 1 What can be extracted by an efficient extractor from the transcript;

Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{P}_{IP}$ convinces us, the instance x is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness w for x ;
- 3 This motivates the security notion of *knowledge soundness*!
- 4 But how can we capture the *knowledge* of $\tilde{P}_{IP}$?

What it knows for sure:

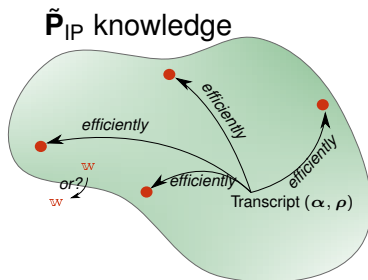


- 1 What can be extracted by an efficient extractor from the transcript;
- 2 So the verifier can extract the witness?

Knowledge soundness

- 1 Soundness ensures that if any (malicious) prover $\tilde{P}_{IP}$ convinces us, the instance x is in the language $L(R)$;
- 2 However, it does not imply that the prover *knows* a witness w for x ;
- 3 This motivates the security notion of *knowledge soundness*!
- 4 But how can we capture the *knowledge* of $\tilde{P}_{IP}$?

What it knows for sure:



- 1 What can be extracted by an efficient extractor from the transcript;
- 2 So the verifier can extract the witness?
- 3 We something stronger than *efficient*: the *rewinding oracle* access to $\tilde{P}_{IP}$.

Knowledge soundness

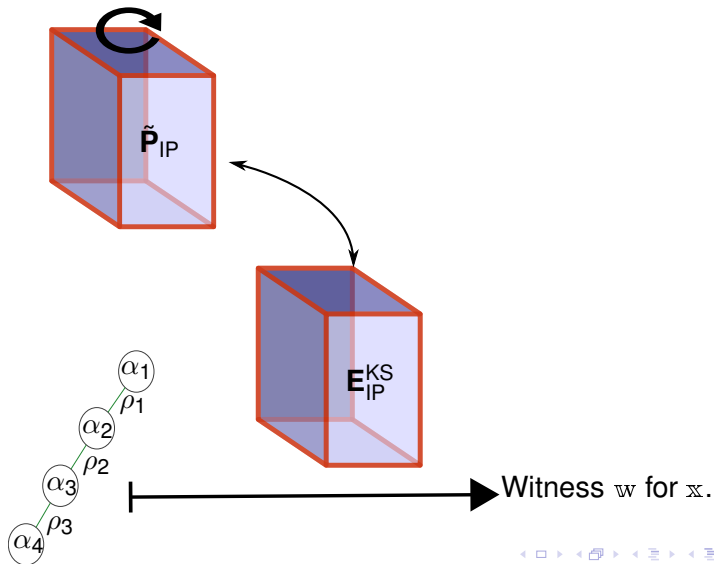
Definition

A public-coin interactive proof $IP = (\mathbf{P}_{IP}, \mathbf{V}_{IP})$ with round complexity k for a relation R has *knowledge soundness error*, κ_{IP} with *extraction time* $et_{IP}(\mathbb{X}, \tilde{\mathbf{P}}_{IP})$ if there exists a probabilistic algorithm $\mathbf{E}_{IP}^{\text{SKS}}$ (the extractor) such that the following holds: for every instance $\mathbb{X}$ and malicious prover $\tilde{\mathbf{P}}_{IP}$ we have

$$\Pr \left[\begin{array}{c} (\mathbb{X}, \mathbb{W}) \notin R \\ \wedge b = 1 \end{array} \middle| \begin{array}{c} \xi \leftarrow \mathcal{X} \\ b \xleftarrow{(\alpha, \rho)} \langle \tilde{\mathbf{P}}_{IP}(\mathbb{X}; \xi), \mathbf{V}_{IP}(\mathbb{X}) \rangle_{IP} \\ \mathbb{W} \leftarrow \mathbf{E}_{IP}^{\text{SKS}}(\mathbb{X}, \alpha, \rho, \tilde{\mathbf{P}}_{IP}(\xi)) \end{array} \right] \leq \kappa_{IP}(\mathbb{X}) .$$

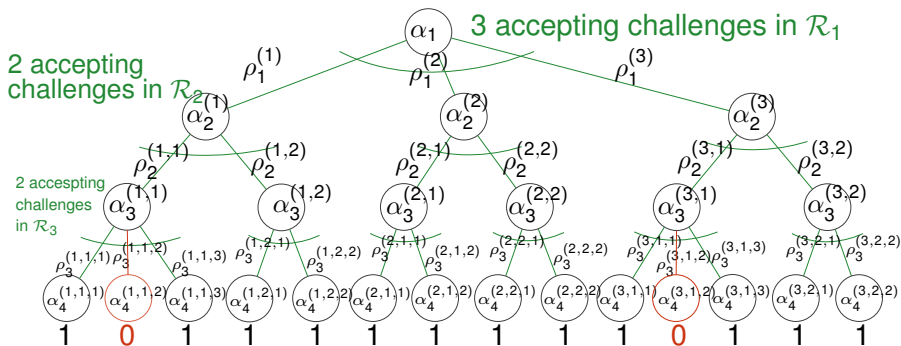
Additionally, $\mathbf{E}_{IP}^{\text{SKS}}(\mathbb{X}, \alpha, \rho, \tilde{\mathbf{P}}_{IP}(\xi))$ runs in expected time $et_{IP}(\mathbb{X}, \tilde{\mathbf{P}}_{IP})$ over ξ, ρ and ν .

Knowledge soundness



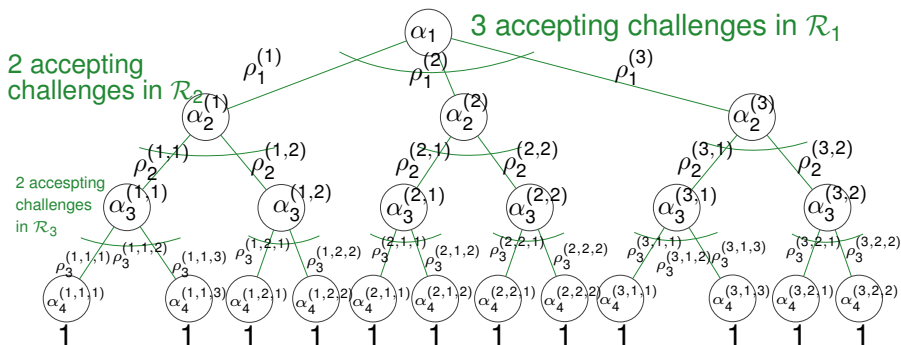
Tree of transcripts

Example of a tree of accepting transcripts:



Tree of transcripts

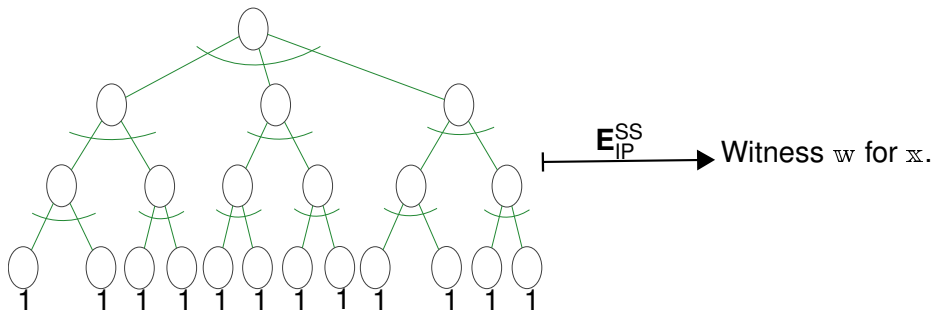
Example of a $(3, 2, 2)$ -tree of accepting transcripts for an instance $\mathbb{x}$:



Special soundness

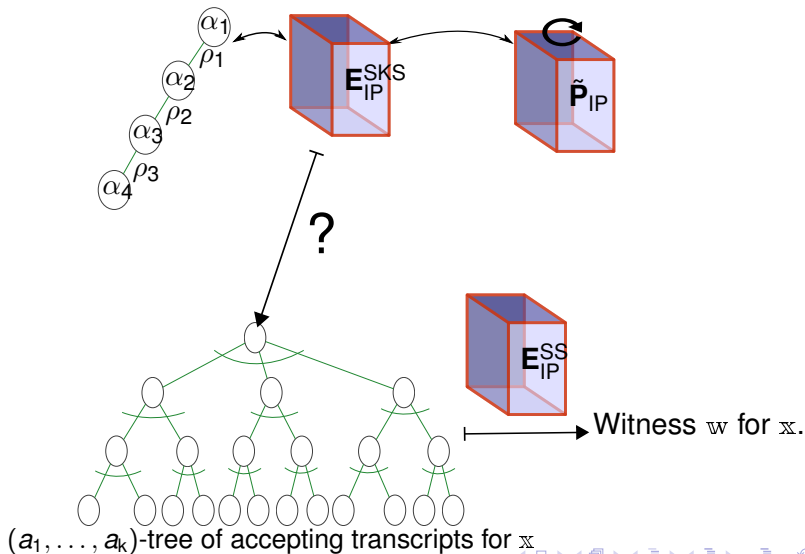
Definition

A public-coin interactive proof $IP = (\mathbf{P}_{IP}, \mathbf{V}_{IP})$ with round complexity k for a relation R has $(a_1, \dots, a_k)$ -*special soundness* if there exists a deterministic polynomial time algorithm $\mathbf{E}_{IP}^{SS}$ (the extractor) such that for every $\mathbb{x}$ and $(a_1, \dots, a_k)$ -tree T of accepting transcripts for $\mathbb{x}$ we have $(\mathbb{x}, \mathbf{E}_{IP}^{SS}(\mathbb{x}, T)) \in R$.

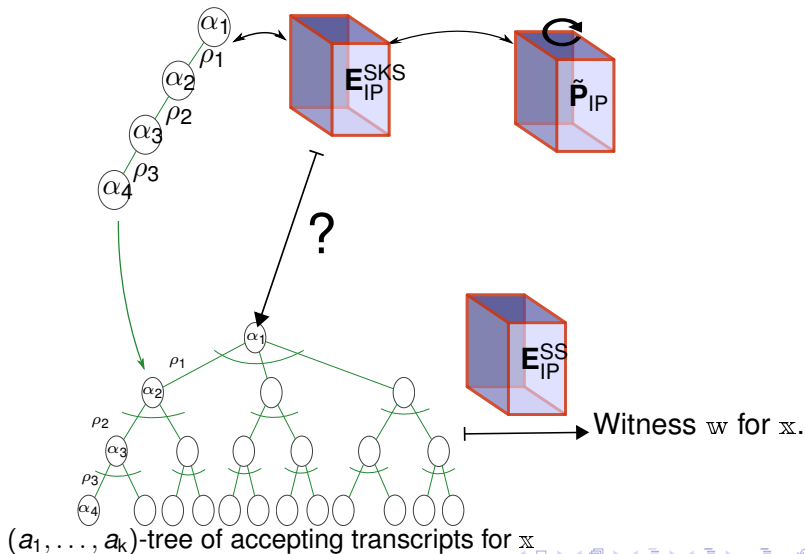


Knowledge soundness from special soundness

Problem statement



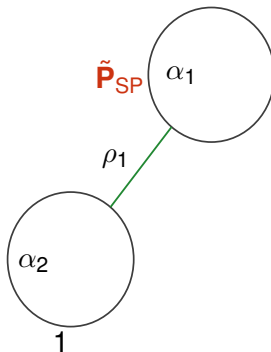
Problem statement



The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

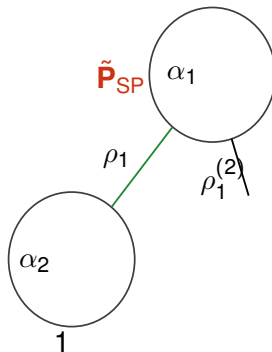


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;

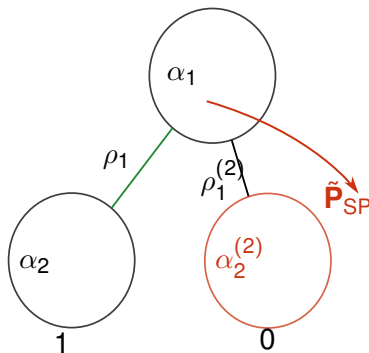


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 2 Test the challenge $\rho_1^{(2)}$;

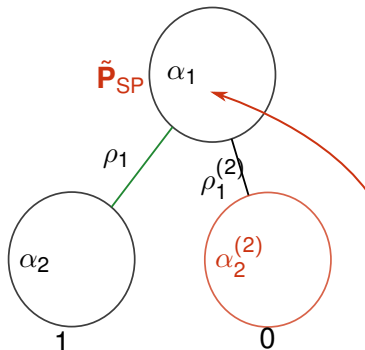


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 2 Test the challenge $\rho_1^{(2)}$;
- 3 Rewind the prover; and continue until we have found a a -tree of accepting transcript;

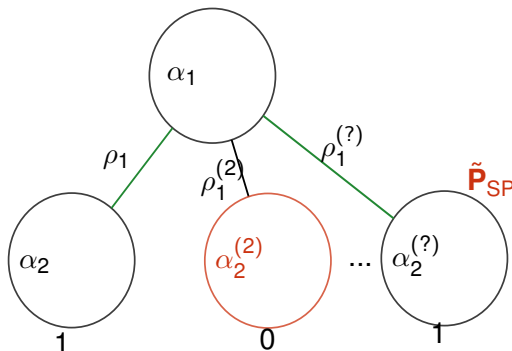


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 2 Test the challenge $\rho_1^{(2)}$;
- 3 Rewind the prover; and continue until we have found a a -tree of accepting transcript;



Early abort strategy

We need one more ingredient.

Early abort strategy

We need one more ingredient.

- Let $b = \mathbf{V}_{\text{SP}}(\mathbb{x}, \alpha_1, \rho, \alpha_2)$ where $(\alpha_1, \rho, \alpha_2)$ is the transcript given to the extractor;

Early abort strategy

We need one more ingredient.

- Let $b = \mathbf{V}_{\text{SP}}(\mathbb{x}, \alpha_1, \rho, \alpha_2)$ where $(\alpha_1, \rho, \alpha_2)$ is the transcript given to the extractor;
- If $b = 1$, run the previous experiment;

Early abort strategy

We need one more ingredient.

- Let $b = \mathbf{V}_{\text{SP}}(\mathbb{x}, \alpha_1, \rho, \alpha_2)$ where $(\alpha_1, \rho, \alpha_2)$ is the transcript given to the extractor;
- If $b = 1$, run the previous experiment;
- However, if $b = 0$, abort;

Early abort strategy

We need one more ingredient.

- Let $b = \mathbf{V}_{\text{SP}}(\mathbb{x}, \alpha_1, \rho, \alpha_2)$ where $(\alpha_1, \rho, \alpha_2)$ is the transcript given to the extractor;
- If $b = 1$, run the previous experiment;
- However, if $b = 0$, abort;
- This is known as the *early abort strategy*.

Extractor expected running time analysis

We show that

Lemma

Let I be the number of invocations done by the extractor to the malicious prover. We have

$$\mathbb{E} \left[I(\rho, \nu) \mid \begin{array}{l} \nu \leftarrow \mathcal{N} \\ \rho \leftarrow \mathcal{R} \\ \mathbf{b} \xleftarrow{((\alpha_1, \alpha_2), \rho)} \langle \tilde{\mathbf{P}}_{\text{SP}}(\mathbb{X}), \mathbf{V}_{\text{SP}}(\mathbb{X}; \rho) \rangle_{\text{SP}} \\ \mathbb{W} \leftarrow \mathbf{E}_{\text{SP}}^{\text{SKS}}(\mathbb{X}, \alpha_1, \rho, \alpha_2, \tilde{\mathbf{P}}_{\text{SP}}; \nu) \end{array} \right] \leq a - 1 .$$

which implies that

$$\text{et}_{\text{SP}}(\mathbb{X}, \tilde{\mathbf{P}}_{\text{SP}}) \leq (a - 1) \cdot \left(\tau_{\tilde{\mathbf{P}}_{\text{SP}}} + \text{poly}(|\mathbb{X}|) \right) .$$

Running time analysis

Proof.

Fix $\mathbb{x}$ arbitrarily and let Acc be the set of challenges that makes the verifier accept. We have

$$\mathbb{E}[I(\rho, \nu)]$$



Running time analysis

Proof.

Fix $\mathbb{x}$ arbitrarily and let Acc be the set of challenges that makes the verifier accept. We have

$$\begin{aligned} & \mathbb{E} [I(\rho, \nu)] \\ &= \mathbb{E} [I(\rho, \nu) \text{ conditioned on } b = 1] \cdot \underbrace{\Pr [b = 1]}_{=|\text{Acc}|/2^r} \end{aligned}$$



Running time analysis

Proof.

Fix $\mathbb{x}$ arbitrarily and let Acc be the set of challenges that makes the verifier accept. We have

$$\begin{aligned} & \mathbb{E}[I(\rho, \nu)] \\ &= \mathbb{E}[I(\rho, \nu) \text{ conditioned on } b = 1] \cdot \underbrace{\Pr[b = 1]}_{=|\text{Acc}|/2^r} \\ &+ \underbrace{\mathbb{E}[I(\rho, \nu) \text{ conditioned on } b = 0]}_{0 \text{ because the algorithm will directly abort.}} \cdot \Pr[b = 0] \end{aligned}$$



Running time analysis

Proof.

Fix $\mathbb{x}$ arbitrarily and let Acc be the set of challenges that makes the verifier accept. We have

$$\begin{aligned} & \mathbb{E} [I(\rho, \nu)] \\ &= \mathbb{E} [I(\rho, \nu) \text{ conditioned on } b = 1] \cdot \underbrace{\Pr [b = 1]}_{=|\text{Acc}|/2^r} \\ & \quad + \underbrace{\mathbb{E} [I(\rho, \nu) \text{ conditioned on } b = 0]}_{0 \text{ because the algorithm will directly abort.}} \cdot \Pr [b = 0] \\ &= \frac{|\text{Acc}|}{2^r} \mathbb{E} [I(\rho, \nu) \text{ conditioned on } b = 1] . \end{aligned}$$



Running time analysis

Proof.

Fix $\mathbb{x}$ arbitrarily and let Acc be the set of challenges that makes the verifier accept. We have

$$\begin{aligned} & \mathbb{E}[I(\rho, \nu)] \\ &= \mathbb{E}[I(\rho, \nu) \text{ conditioned on } b = 1] \cdot \underbrace{\Pr[b = 1]}_{=|\text{Acc}|/2^r} \\ & \quad + \underbrace{\mathbb{E}[I(\rho, \nu) \text{ conditioned on } b = 0]}_{0 \text{ because the algorithm will directly abort.}} \cdot \Pr[b = 0] \\ &= \frac{|\text{Acc}|}{2^r} \mathbb{E}[I(\rho, \nu) \text{ conditioned on } b = 1] . \end{aligned}$$

Let

$$e := \mathbb{E}[I(\rho, \nu) \text{ conditioned on } b = 1] .$$

I claim that $e \leq (a - 1) \frac{2^r}{|\text{Acc}|}$, which would conclude the proof of lemma. □

Running time analysis

Proof.

We consider two distinct cases:

.



Running time analysis

Proof.

We consider two distinct cases:

- $|\mathbf{Acc}| < a$. There are $2^r - 1$ invocations, which is smaller than $(a - 1) \frac{2^r}{|\mathbf{Acc}|}$.



Running time analysis

Proof.

We consider two distinct cases:

- $|\mathbf{Acc}| < a$. There are $2^r - 1$ invocations, which is smaller than $(a - 1) \frac{2^r}{|\mathbf{Acc}|}$.
- $|\mathbf{Acc}| \geq a$. The extractor runs until it has found $a - 1$ accepting challenges.

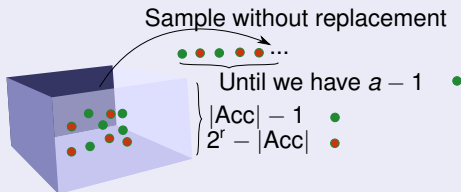


Running time analysis

Proof.

We consider two distinct cases:

- $|\mathbf{Acc}| < a$. There are $2^r - 1$ invocations, which is smaller than $(a - 1) \frac{2^r}{|\mathbf{Acc}|}$.
- $|\mathbf{Acc}| \geq a$. The extractor runs until it has found $a - 1$ accepting challenges.

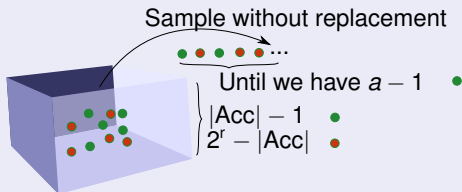


Running time analysis

Proof.

We consider two distinct cases:

- $|\mathbf{Acc}| < a$. There are $2^r - 1$ invocations, which is smaller than $(a - 1) \frac{2^r}{|\mathbf{Acc}|}$.
- $|\mathbf{Acc}| \geq a$. The extractor runs until it has found $a - 1$ accepting challenges.



This is known as the *Negative hypergeometric distribution*, and its expectation is given by $(a - 1) \frac{2^r}{|\mathbf{Acc}|}$. □

Knowledge soundness analysis

Lemma

The following holds for the extractor $\mathbf{E}_{\text{SP}}^{\text{SKS}}$ described above:

$$\Pr \left[\begin{array}{c} (\mathbb{X}, \mathbb{W}) \notin R \\ \wedge \mathbf{b} = 1 \end{array} \middle| \begin{array}{c} \rho \xleftarrow{\quad} \mathcal{R} \\ \mathbf{b} \xleftarrow{((\alpha_1, \alpha_2), \rho)} \langle \tilde{\mathbf{P}}_{\text{SP}}(\mathbb{X}), \mathbf{V}_{\text{SP}}(\mathbb{X}; \rho) \rangle_{\text{SP}} \\ \mathbb{W} \xleftarrow{\quad} \mathbf{E}_{\text{SP}}^{\text{SKS}}(\mathbb{X}, \alpha_1, \rho, \alpha_2, \tilde{\mathbf{P}}_{\text{SP}}) \end{array} \right] \leq \frac{a-1}{2^r},$$

where a is the arity of the special soundness and $\mathcal{R} = \{0, 1\}^r$.

Knowledge soundness analysis

Proof.

We want

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \middle| \begin{array}{c} \rho \xleftarrow{\quad} \mathcal{R} \\ b \xleftarrow{((\alpha_1, \alpha_2), \rho)} \langle \tilde{\mathbf{P}}_{\text{SP}}(\mathbb{x}), \mathbf{V}_{\text{SP}}(\mathbb{x}; \rho) \rangle_{\text{SP}} \\ \mathbb{w} \xleftarrow{\quad} \mathbf{E}_{\text{SP}}^{\text{SKS}}(\mathbb{x}, \alpha_1, \rho, \alpha_2, \tilde{\mathbf{P}}_{\text{SP}}) \end{array} \right] \leq \frac{a-1}{2^r} .$$

Fix $\mathbb{x}$ arbitrarily. We consider two distinct cases:



Knowledge soundness analysis

Proof.

We want

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \middle| \begin{array}{c} \rho \xleftarrow{\mathcal{R}} \\ b \xleftarrow{((\alpha_1, \alpha_2), \rho)} \\ \mathbb{w} \xleftarrow{\langle \tilde{\mathbf{P}}_{\text{SP}}(\mathbb{x}), \mathbf{V}_{\text{SP}}(\mathbb{x}; \rho) \rangle_{\text{SP}} \\ \mathbf{E}_{\text{SP}}^{\text{SKS}}(\mathbb{x}, \alpha_1, \rho, \alpha_2, \tilde{\mathbf{P}}_{\text{SP}})} \end{array} \right] \leq \frac{a-1}{2^r} .$$

Fix $\mathbb{x}$ arbitrarily. We consider two distinct cases:

- **|Acc| ≥ a.** If $b = 0$, the probability is zero. Otherwise, the extractor succeeds almost surely since it passes the early abort test, and therefore, the probability is zero.



Knowledge soundness analysis

Proof.

We want

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge \mathbf{b} = \mathbf{1} \end{array} \middle| \begin{array}{ccc} \rho & \leftarrow & \mathcal{R} \\ \mathbf{b} & \xleftarrow{((\alpha_1, \alpha_2), \rho)} & \langle \tilde{\mathbf{P}}_{\text{SP}}(\mathbb{x}), \mathbf{V}_{\text{SP}}(\mathbb{x}; \rho) \rangle_{\text{SP}} \\ \mathbb{w} & \leftarrow & \mathbf{E}_{\text{SP}}^{\text{SKS}}(\mathbb{x}, \alpha_1, \rho, \alpha_2, \tilde{\mathbf{P}}_{\text{SP}}) \end{array} \right] \leq \frac{a-1}{2^r} .$$

Fix $\mathbb{x}$ arbitrarily. We consider two distinct cases:

- **|Acc| ≥ a.** If $\mathbf{b} = \mathbf{0}$, the probability is zero. Otherwise, the extractor succeeds almost surely since it passes the early abort test, and therefore, the probability is zero.
- **|Acc| < a.** The extractor cannot find a **(a)**-tree of transcripts for $\mathbb{x}$; therefore we just looking at the probability that $\mathbf{b} = \mathbf{1}$, which is $\frac{|\text{Acc}|}{2^r} \leq \frac{a-1}{2^r}$.



The result

Theorem

Let $\text{SP} = (\mathbf{P}_{\text{SP}}, \mathbf{V}_{\text{SP}})$ be a sigma protocol (SP) for a relation R with special soundness with arity a and verifier randomness complexity r . It follows that SP has strong rewinding knowledge soundness error

$$\kappa_{\text{SP}}(\mathbb{X}) \leq \frac{a-1}{2^r}$$

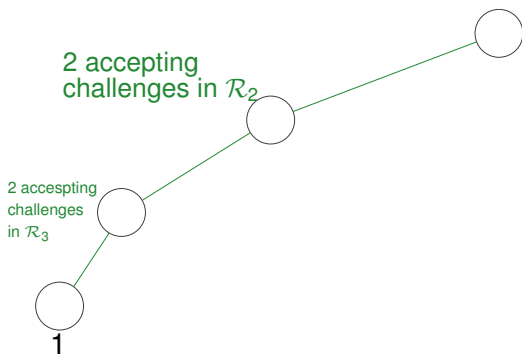
and extraction time

$$\text{et}_{\text{SP}}(\mathbb{X}, \tilde{\mathbf{P}}_{\text{SP}}) \leq (a-1) \cdot \left(\tau_{\tilde{\mathbf{P}}_{\text{SP}}} + \text{poly}(|\mathbb{X}|) \right) .$$

Moreover, the number of times the knowledge soundness extractor invokes $\tilde{\mathbf{P}}_{\text{IP}}$ is $a-1$.

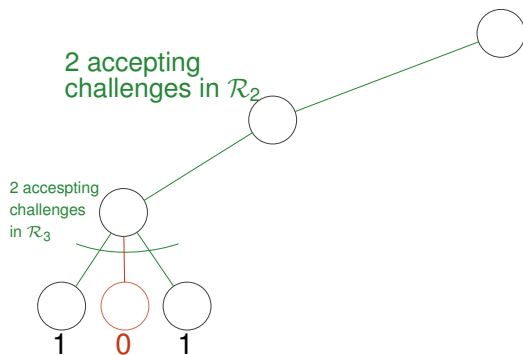
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



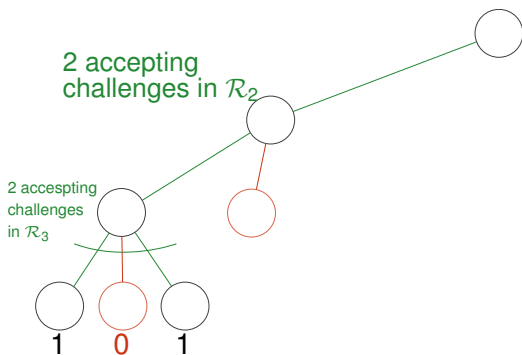
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



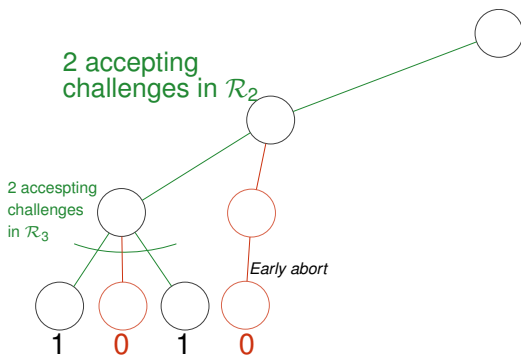
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



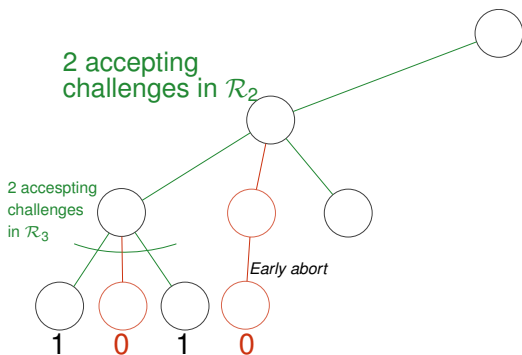
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



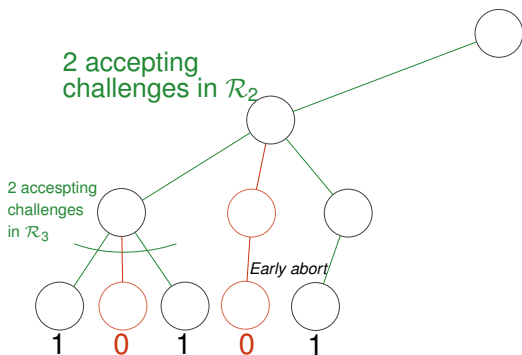
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



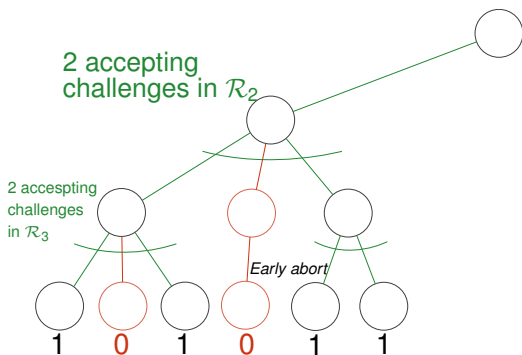
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



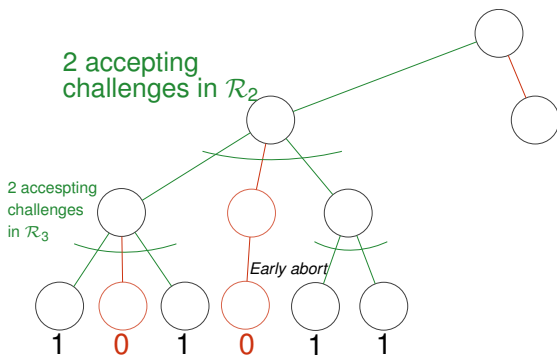
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



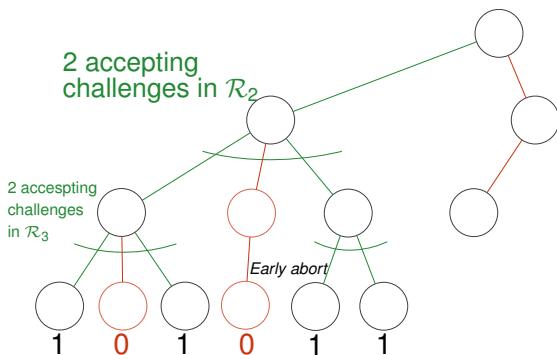
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



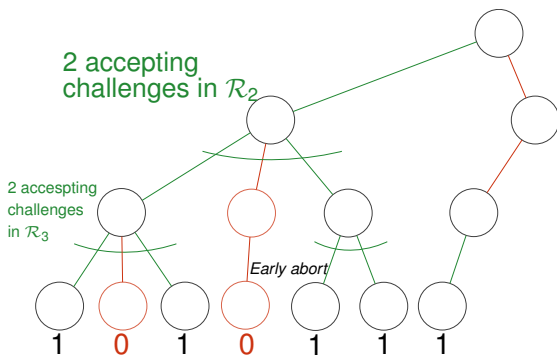
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



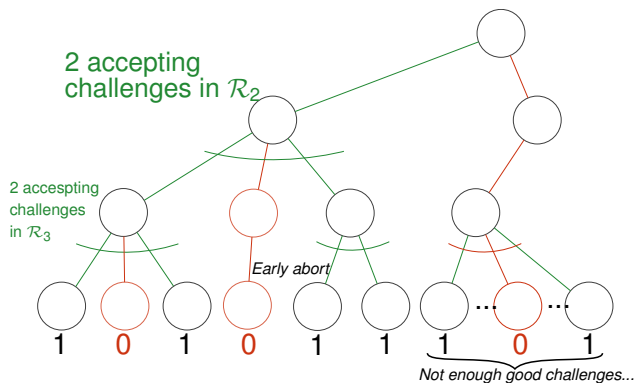
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



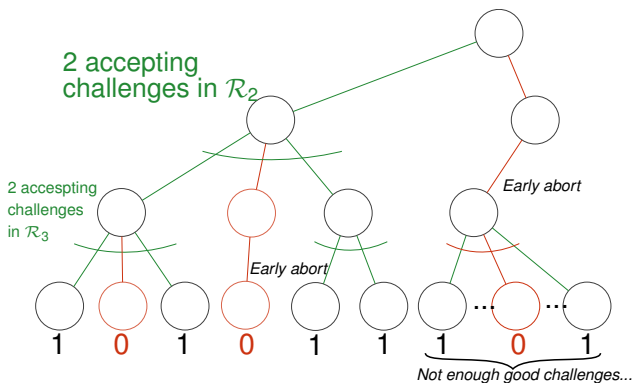
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



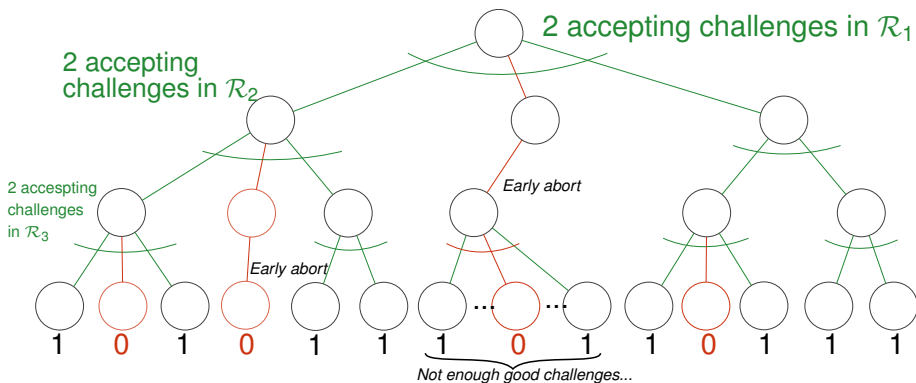
How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



How to generalize to IPs

Suppose we are searching for a $(2, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$.



The result

Theorem

Let $\text{IP} = (\mathbf{P}_{\text{IP}}, \mathbf{V}_{\text{IP}})$ be a public-coin interactive proof for a relation R with round complexity k , $(a_1, \dots, a_k)$ -special soundness and randomness complexity $(r_1, \dots, r_k)$. Then IP has strong rewinding knowledge soundness error

$$\kappa_{\text{IP}}(\mathbb{X}) \leq 1 - \prod_{i=1}^k \left(1 - \frac{a_i - 1}{2^{r_i}}\right)$$

and extraction time

$$\text{et}_{\text{IP}}(\mathbb{X}, \tilde{\mathbf{P}}_{\text{IP}}) \leq \left(\left(\prod_{i=1}^k a_i \right) - 1 \right) \cdot \left(\tau_{\tilde{\mathbf{P}}_{\text{IP}}} + \text{poly}(|\mathbb{X}|) \right) .$$

Moreover, the number of times the knowledge soundness extractor invokes $\tilde{\mathbf{P}}_{\text{IP}}$ is $\left(\prod_{i=1}^k a_i \right) - 1$.

State restoration

Fiat–Shamir transformation

$$\tilde{\mathbf{p}}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X}; \xi) \\ \alpha_1 \leftarrow \tilde{\mathbf{p}}_{\text{IP}}(\mathbb{X}; \xi)$$

$$\vdots \\ \alpha_k \leftarrow \tilde{\mathbf{p}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_{k-1}), \\ (\rho_1, \dots, \rho_{k-1}); \xi)$$

$$\alpha_{k+1} \leftarrow \tilde{\mathbf{p}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_k), \\ (\rho_1, \dots, \rho_k); \xi)$$

$$\mathbf{v}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X})$$

Fiat–Shamir transformation

$$\tilde{\mathbf{P}}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X}; \xi)$$
$$\alpha_1 \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}; \xi)$$

$$\rho_1 := \text{rnd}_1(\mathbb{X}, (\alpha_1))$$

$$\vdots$$

$$\alpha_k \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_{k-1}),$$
$$(\rho_1, \dots, \rho_{k-1}); \xi)$$

$$\alpha_{k+1} \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_k),$$
$$(\rho_1, \dots, \rho_k); \xi)$$

$$\mathbf{V}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X})$$

Fiat–Shamir transformation

$$\tilde{\mathbf{P}}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X}; \xi)$$
$$\alpha_1 \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}; \xi)$$

$$\rho_1 := \text{rnd}_1(\mathbb{X}, (\alpha_1))$$

$$\vdots$$

$$\alpha_k \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_{k-1}),$$
$$(\rho_1, \dots, \rho_{k-1}); \xi)$$

$$\rho_k := \text{rnd}_k(\mathbb{X}, (\alpha_1, \dots, \alpha_k))$$

$$\alpha_{k+1} \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_k),$$
$$(\rho_1, \dots, \rho_k); \xi)$$

$$\mathbf{V}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X})$$

Fiat–Shamir transformation

$$\tilde{\mathbf{P}}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X}; \xi) \\ \alpha_1 \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}; \xi)$$

$$\rho_1 := \text{rnd}_1(\mathbb{X}, (\alpha_1))$$

$$\vdots$$

$$\alpha_k \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_{k-1}), \\ (\rho_1, \dots, \rho_{k-1}); \xi)$$

$$\rho_k := \text{rnd}_k(\mathbb{X}, (\alpha_1, \dots, \alpha_k))$$

$$\alpha_{k+1} \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_k), \\ (\rho_1, \dots, \rho_k); \xi)$$

$$\pi := ((\alpha_1, \dots, \alpha_k)) \xrightarrow{\pi}$$

$$\mathbf{V}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X})$$

Fiat–Shamir transformation

$$\tilde{\mathbf{P}}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X}; \xi) \\ \alpha_1 \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}; \xi)$$

$$\rho_1 := \text{rnd}_1(\mathbb{X}, (\alpha_1))$$

$$\vdots$$

$$\alpha_k \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_{k-1}), \\ (\rho_1, \dots, \rho_{k-1}); \xi)$$

$$\rho_k := \text{rnd}_k(\mathbb{X}, (\alpha_1, \dots, \alpha_k))$$

$$\alpha_{k+1} \leftarrow \tilde{\mathbf{P}}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_k), \\ (\rho_1, \dots, \rho_k); \xi)$$

$$\pi := ((\alpha_1, \dots, \alpha_k))$$

$$\xrightarrow{\pi}$$

$$\mathbf{V}_{\text{NIROP}}^{\text{rnd}}(\mathbb{X})$$

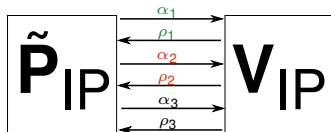
Compute

$\rho_i := \text{rnd}_i(\mathbb{X}, (\alpha_1, \dots, \alpha_k), (\rho_1, \dots, \rho_{k-1}); \xi)$

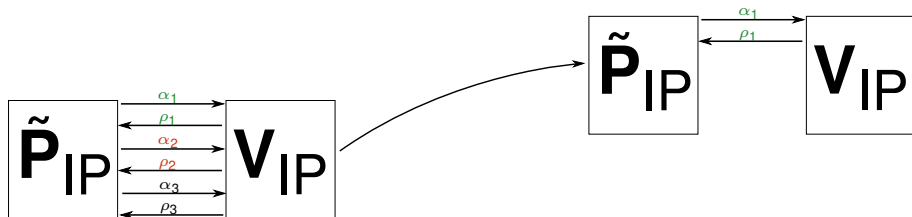
for all $i \in [k+1]$ and check that

$$\mathbf{V}_{\text{IP}}(\mathbb{X}, (\alpha_1, \dots, \alpha_{k+1}), (\rho_1, \dots, \rho_k)) = 1$$

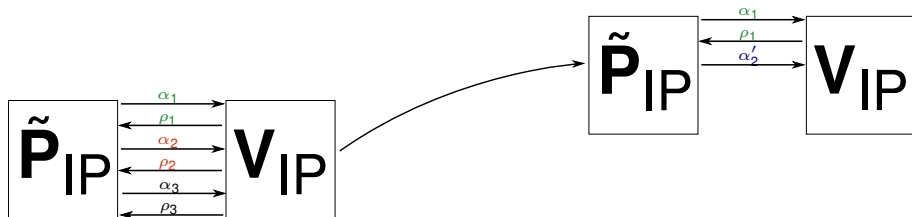
Fiat–Shamir and state restoration



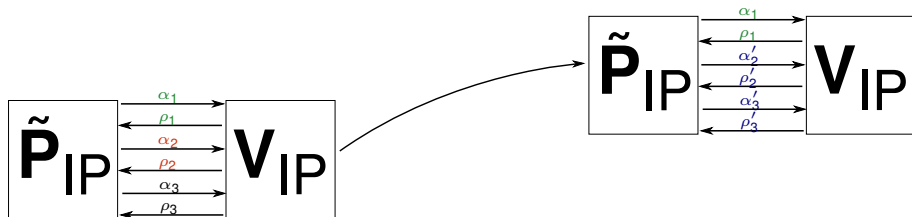
Fiat–Shamir and state restoration



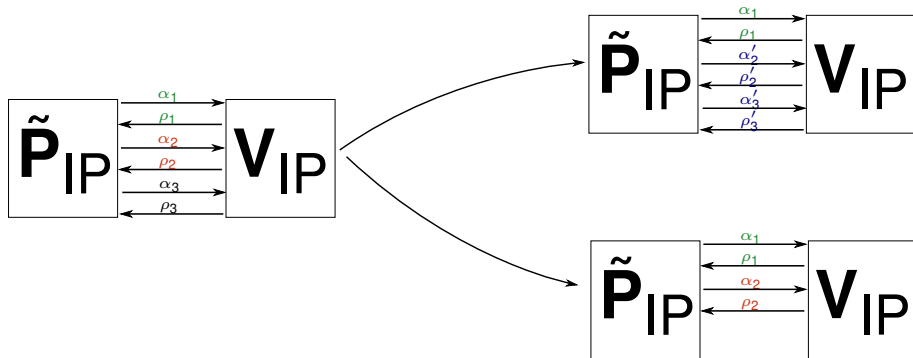
Fiat–Shamir and state restoration



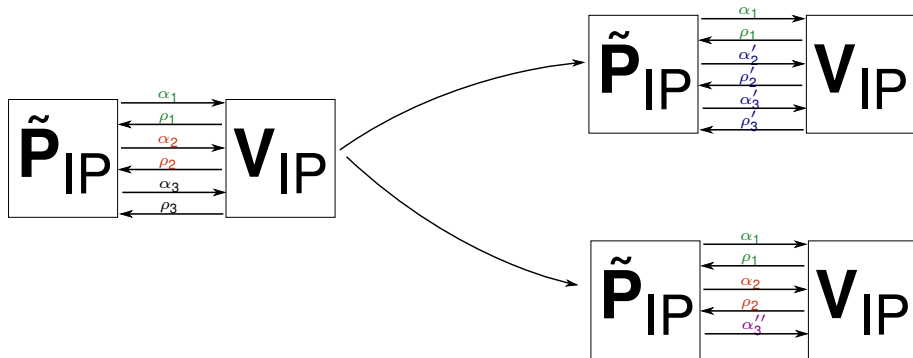
Fiat–Shamir and state restoration



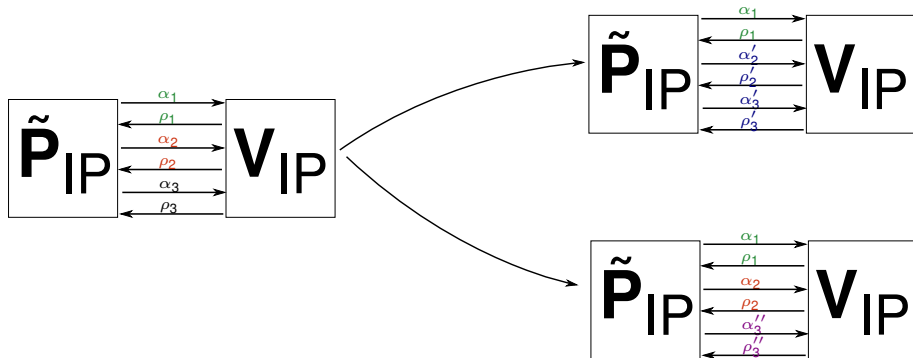
Fiat–Shamir and state restoration



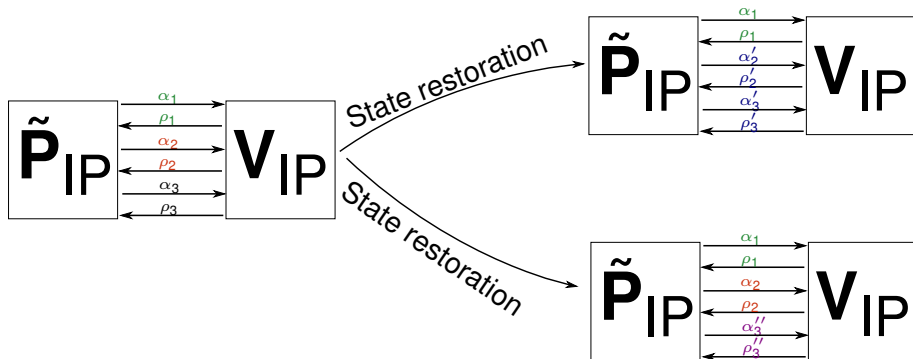
Fiat-Shamir and state restoration



Fiat-Shamir and state restoration



Fiat–Shamir and state restoration



The state restoration game

Definition (State restoration game)

For any ordered tuple of random oracles $(\text{rnd}_1, \dots, \text{rnd}_k)$ and t -queries random-oracle algorithm $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$, the *state restoration game* is:

$$\Gamma^{\text{sr}}((\text{rnd}_1, \dots, \text{rnd}_k), \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}):$$

The state restoration game

Definition (State restoration game)

For any ordered tuple of random oracles $(\text{rnd}_1, \dots, \text{rnd}_k)$ and t -queries random-oracle algorithm $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$, the *state restoration game* is:

$\Gamma^{\text{sr}}((\text{rnd}_1, \dots, \text{rnd}_k), \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}):$

- 1 Run the state restoration prover

$$(\mathbb{X}, (\alpha_1, \dots, \alpha_k)) \xleftarrow{(\text{tr}_1^{\text{sr}}, \dots, \text{tr}_k^{\text{sr}})} (\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})^{(\text{rnd}_1, \dots, \text{rnd}_k)}.$$

The state restoration game

Definition (State restoration game)

For any ordered tuple of random oracles $(\text{rnd}_1, \dots, \text{rnd}_k)$ and t -queries random-oracle algorithm $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$, the *state restoration game* is:

$\Gamma^{\text{sr}}((\text{rnd}_1, \dots, \text{rnd}_k), \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}):$

- 1 Run the state restoration prover

$$(\mathbb{X}, (\alpha_1, \dots, \alpha_k)) \xleftarrow{(\text{tr}_1^{\text{sr}}, \dots, \text{tr}_k^{\text{sr}})} (\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})^{(\text{rnd}_1, \dots, \text{rnd}_k)}.$$

- 2 Get challenges $\rho_i := \text{rnd}_i(\mathbb{X}, (\alpha_1, \dots, \alpha_i))$, for every $i \in [k]$.

The state restoration game

Definition (State restoration game)

For any ordered tuple of random oracles $(\text{rnd}_1, \dots, \text{rnd}_k)$ and t -queries random-oracle algorithm $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$, the *state restoration game* is:

$\Gamma^{\text{sr}}((\text{rnd}_1, \dots, \text{rnd}_k), \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}):$

- 1 Run the state restoration prover

$$(\mathbb{X}, (\alpha_1, \dots, \alpha_k)) \xleftarrow{(\text{tr}_1^{\text{sr}}, \dots, \text{tr}_k^{\text{sr}})} (\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})^{(\text{rnd}_1, \dots, \text{rnd}_k)}.$$

- 2 Get challenges $\rho_i := \text{rnd}_i(\mathbb{X}, (\alpha_1, \dots, \alpha_i))$, for every $i \in [k]$.
- 3 Return $(\mathbb{X}, (\alpha_1, \dots, \alpha_k), (\rho_1, \dots, \rho_k))$.

The state restoration game

Definition (State restoration game)

For any ordered tuple of random oracles $(\text{rnd}_1, \dots, \text{rnd}_k)$ and t -queries random-oracle algorithm $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$, the *state restoration game* is:

$\Gamma^{\text{sr}}((\text{rnd}_1, \dots, \text{rnd}_k), \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}):$

1 Run the state restoration prover

$(\mathbb{X}, (\alpha_1, \dots, \alpha_k)) \xleftarrow{(\text{tr}_1^{\text{sr}}, \dots, \text{tr}_k^{\text{sr}})} (\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})^{(\text{rnd}_1, \dots, \text{rnd}_k)}.$

2 Get challenges $\rho_i := \text{rnd}_i(\mathbb{X}, (\alpha_1, \dots, \alpha_i))$, for every $i \in [k]$.

3 Return $(\mathbb{X}, (\alpha_1, \dots, \alpha_k), (\rho_1, \dots, \rho_k)).$

We denote the whole process by

$$(\mathbb{X}, (\alpha_1, \dots, \alpha_k), (\rho_1, \dots, \rho_k)) \xleftarrow{(\text{tr}_1^{\text{sr}}, \dots, \text{tr}_k^{\text{sr}})} \Gamma^{\text{sr}}((\text{rnd}_1, \dots, \text{rnd}_k), \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}).$$

State restoration knowledge soundness

Definition (State restoration knowledge soundness)

IP has *rewinding state restoration knowledge soundness error* $\kappa_{\text{IP}}^{\text{sr}}$ with *extraction time* $\text{et}_{\text{IP}}^{\text{sr}}$ if there exists a probabilistic algorithm $\mathbf{E}_{\text{IP}}^{\text{SRKS}}$ (the extractor) such that for every query bound t , t -query deterministic malicious adaptive prover $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$.

$$\Pr \left[\begin{array}{l} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\mathbb{x}, t) ,$$

Additionally, $\mathbf{E}_{\text{IP}}^{\text{SRKS}}(\mathbb{x}, \alpha, \sigma, \rho, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ runs in expected time $\text{et}_{\text{IP}}^{\text{sr}}(\mathbb{x}, t, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ over its internal randomness.

State restoration knowledge soundness

Definition (State restoration knowledge soundness)

IP has *rewinding state restoration knowledge soundness error* $\kappa_{\text{IP}}^{\text{sr}}$ with *extraction time* $\text{et}_{\text{IP}}^{\text{sr}}$ if there exists a probabilistic algorithm $\mathbf{E}_{\text{IP}}^{\text{SRKS}}$ (the extractor) such that for every query bound t , t -query deterministic malicious adaptive prover $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$.

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \middle| \text{rnd} \leftarrow \mathcal{U}((r_i)_{i \in [k]}) \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\mathbb{x}, t) ,$$

Additionally, $\mathbf{E}_{\text{IP}}^{\text{SRKS}}(\mathbb{x}, \alpha, \sigma, \rho, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ runs in expected time $\text{et}_{\text{IP}}^{\text{sr}}(\mathbb{x}, t, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ over its internal randomness.

State restoration knowledge soundness

Definition (State restoration knowledge soundness)

IP has *rewinding state restoration knowledge soundness error* $\kappa_{\text{IP}}^{\text{sr}}$ with *extraction time* $\text{et}_{\text{IP}}^{\text{sr}}$ if there exists a probabilistic algorithm $\mathbf{E}_{\text{IP}}^{\text{SRKS}}$ (the extractor) such that for every query bound t , t -query deterministic malicious adaptive prover $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$.

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \middle| \begin{array}{c} \text{rnd} \xleftarrow{\quad} \mathcal{U}((r_i)_{i \in [k]}) \\ (\mathbb{x}, \alpha, \rho) \xleftarrow{\text{tr}^{\text{sr}}} \Gamma^{\text{sr}}(\text{rnd}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}) \end{array} \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\mathbb{x}, t) ,$$

Additionally, $\mathbf{E}_{\text{IP}}^{\text{SRKS}}(\mathbb{x}, \alpha, \sigma, \rho, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ runs in expected time $\text{et}_{\text{IP}}^{\text{sr}}(\mathbb{x}, t, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ over its internal randomness.

State restoration knowledge soundness

Definition (State restoration knowledge soundness)

IP has *rewinding state restoration knowledge soundness error* $\kappa_{\text{IP}}^{\text{sr}}$ with *extraction time* $\text{et}_{\text{IP}}^{\text{sr}}$ if there exists a probabilistic algorithm $\mathbf{E}_{\text{IP}}^{\text{SRKS}}$ (the extractor) such that for every query bound t , t -query deterministic malicious adaptive prover $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$.

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \middle| \begin{array}{c} \text{rnd} \leftarrow \mathcal{U}((r_i)_{i \in [k]}) \\ (\mathbb{x}, \alpha, \rho) \xleftarrow{\text{tr}^{\text{sr}}} \Gamma^{\text{sr}}(\text{rnd}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}) \\ b \leftarrow \mathbf{V}_{\text{IP}}(\mathbb{x}, \alpha, \rho) \end{array} \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\mathbb{x}, t) ,$$

Additionally, $\mathbf{E}_{\text{IP}}^{\text{SRKS}}(\mathbb{x}, \alpha, \sigma, \rho, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ runs in expected time $\text{et}_{\text{IP}}^{\text{sr}}(\mathbb{x}, t, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ over its internal randomness.

State restoration knowledge soundness

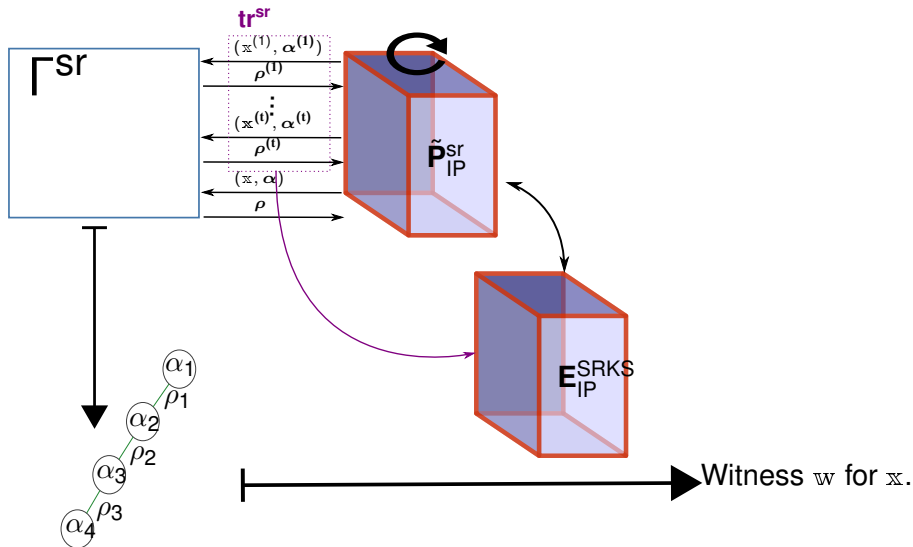
Definition (State restoration knowledge soundness)

IP has *rewinding state restoration knowledge soundness error* $\kappa_{\text{IP}}^{\text{sr}}$ with *extraction time* $\text{et}_{\text{IP}}^{\text{sr}}$ if there exists a probabilistic algorithm $\mathbf{E}_{\text{IP}}^{\text{SRKS}}$ (the extractor) such that for every query bound t , t -query deterministic malicious adaptive prover $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$.

$$\Pr \left[\begin{array}{c} (\mathbb{x}, \mathbb{w}) \notin R \\ \wedge b = 1 \end{array} \middle| \begin{array}{c} \text{rnd} \leftarrow \mathcal{U}((r_i)_{i \in [k]}) \\ (\mathbb{x}, \alpha, \rho) \xleftarrow{\text{tr}^{\text{sr}}} \Gamma^{\text{sr}}(\text{rnd}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}) \\ b \leftarrow \mathbf{V}_{\text{IP}}(\mathbb{x}, \alpha, \rho) \\ \mathbb{w} \leftarrow \mathbf{E}_{\text{IP}}^{\text{SRKS}}(\mathbb{x}, \alpha, \rho, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}) \end{array} \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\mathbb{x}, t) ,$$

Additionally, $\mathbf{E}_{\text{IP}}^{\text{SRKS}}(\mathbb{x}, \alpha, \sigma, \rho, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ runs in expected time $\text{et}_{\text{IP}}^{\text{sr}}(\mathbb{x}, t, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}})$ over its internal randomness.

State restoration knowledge soundness



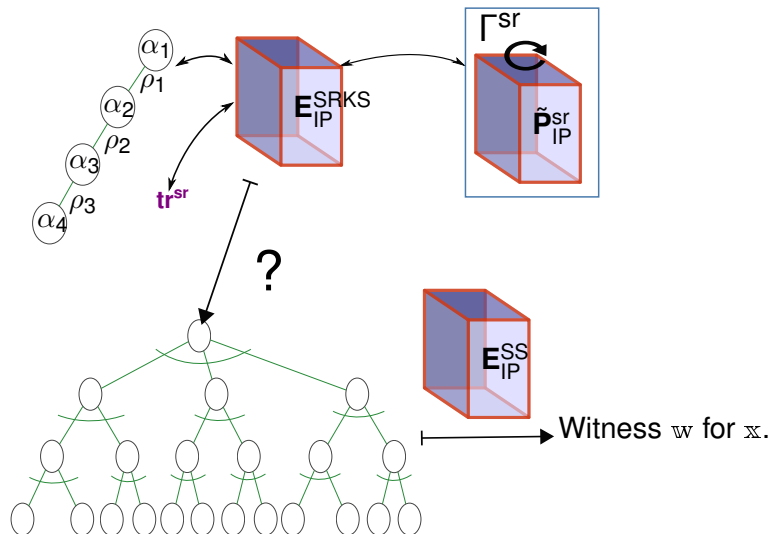
Knowledge soundness of the non-interactive protocol from state restoration knowledge soundness

Theorem (Informal)

If IP has state restoration knowledge soundness $\kappa_{\text{IP}}^{\text{sr}}$ and extraction time $\text{et}_{\text{IP}}^{\text{sr}}$, then its non-interactive Fiat–Shamir transform has the same knowledge soundness and the same extraction time.

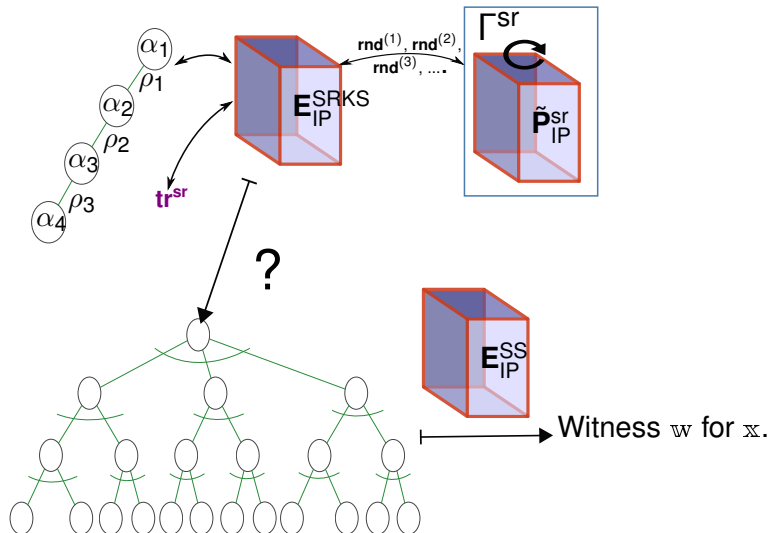
State restoration knowledge soundness from special soundness

Problem statement



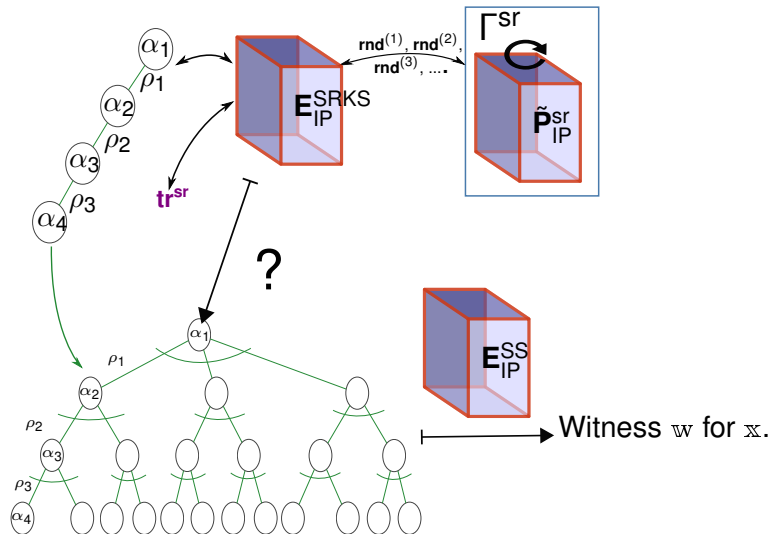
$(a_1, \dots, a_k)$ -tree of accepting transcripts for x

Problem statement



$(a_1, \dots, a_k)$ -tree of accepting transcripts for $\mathbb{X}$

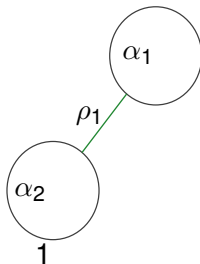
Problem statement



$(a_1, \dots, a_k)$ -tree of accepting transcripts for $\mathbb{X}$

The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.
Extractor:

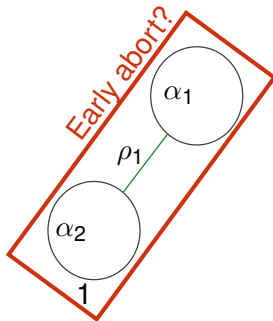


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;

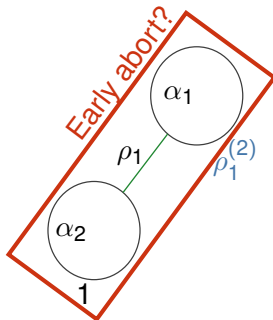


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;

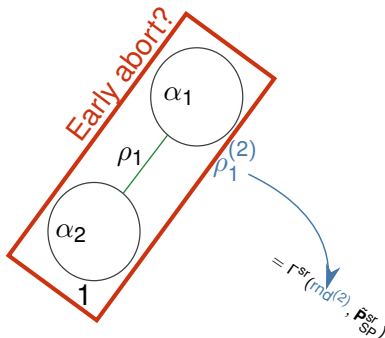


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that...;

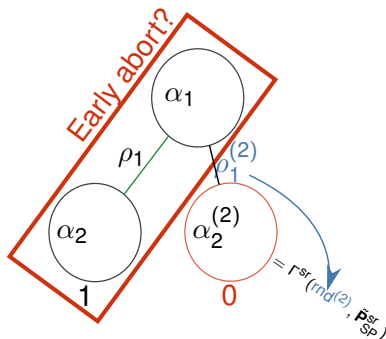


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that...;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ and the verifier accepts;

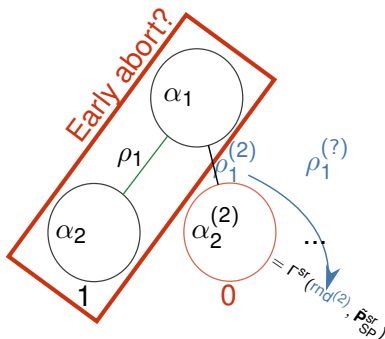


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that...;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ and the verifier accepts;
- 5 Continue until we have found a good challenges;

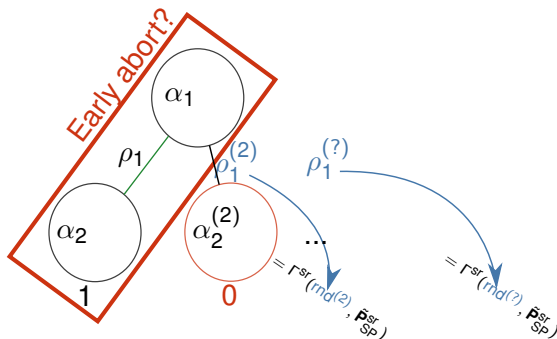


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that...;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ and the verifier accepts;
- 5 Continue until we have found a good challenges;

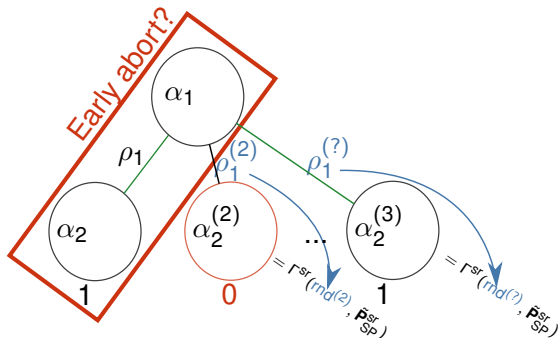


The extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

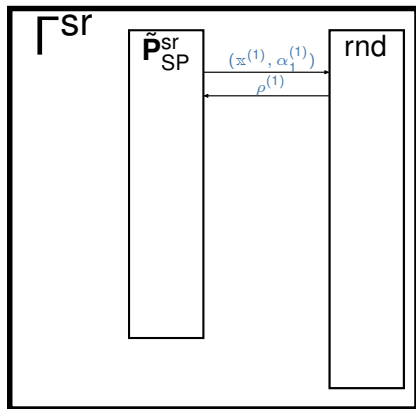
- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that...;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ and the verifier accepts;
- 5 Continue until we have found a good challenges;



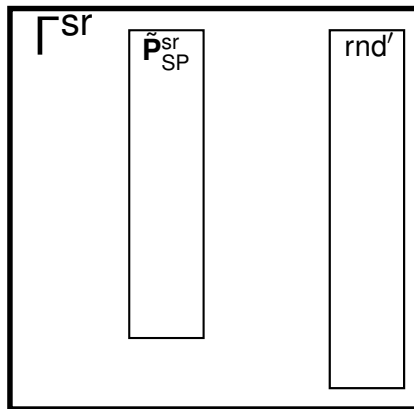
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



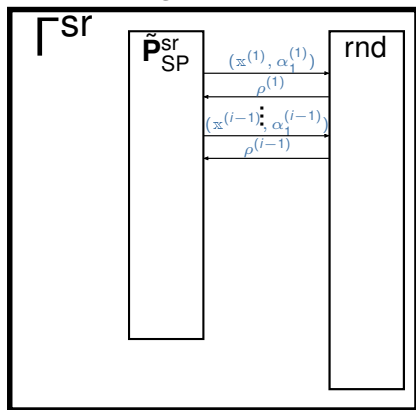
Needed



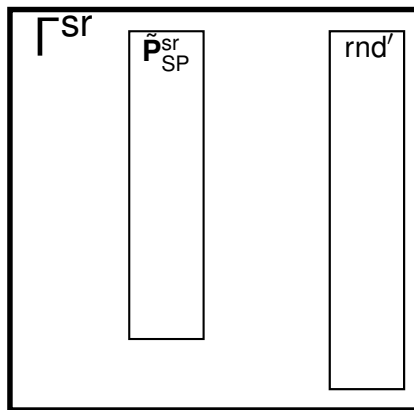
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



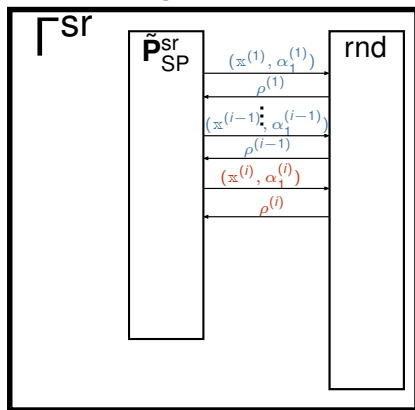
Needed



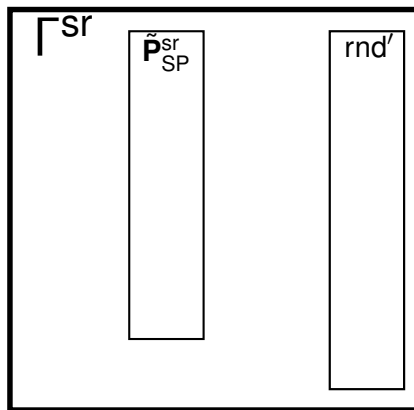
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



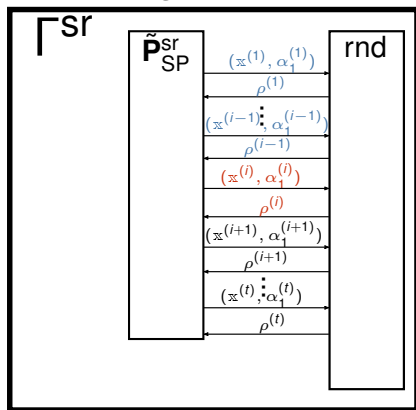
Needed



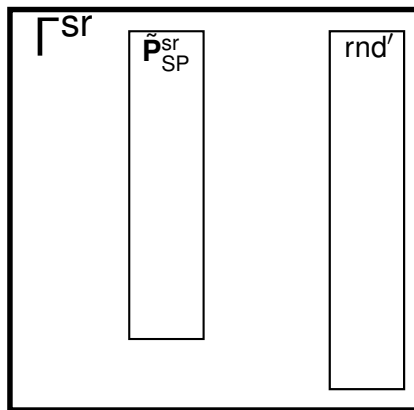
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)'}, \rho^{(i)'}, \cdot)$.

Original run



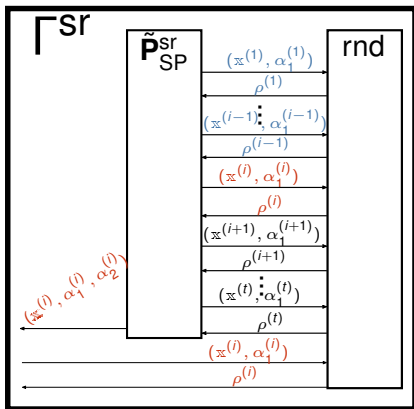
Needed



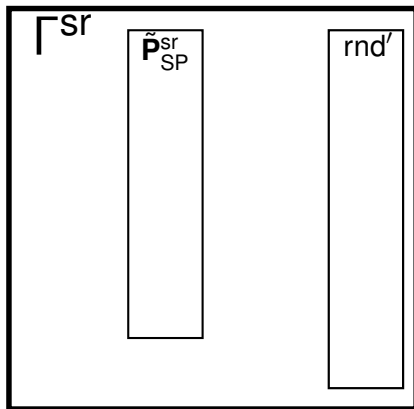
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



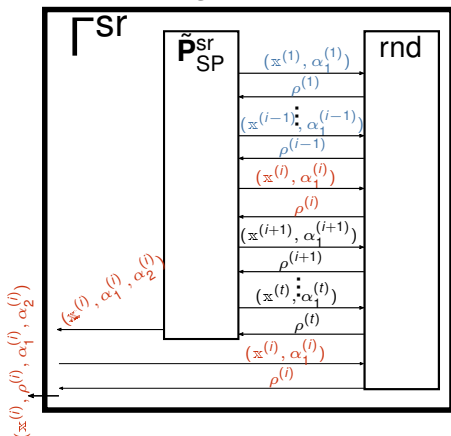
Needed



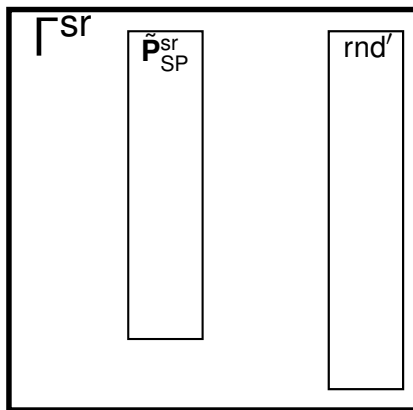
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



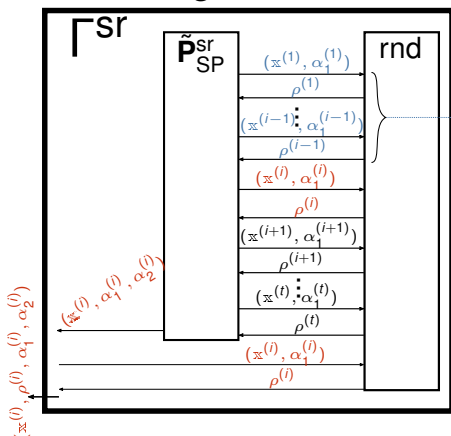
Needed



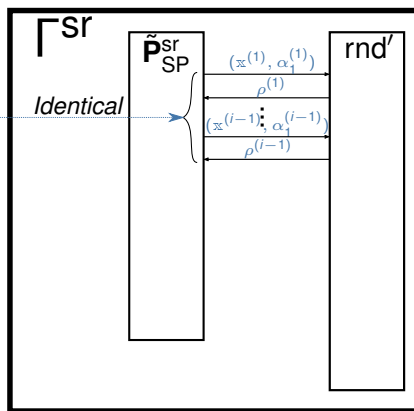
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



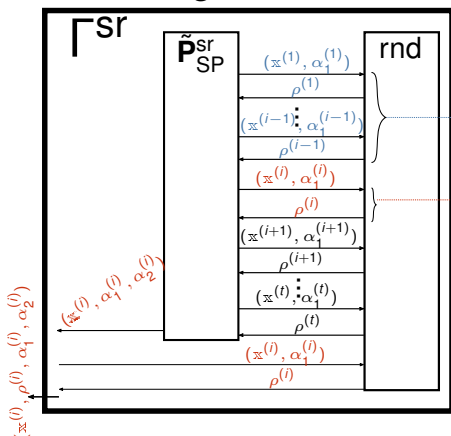
Needed



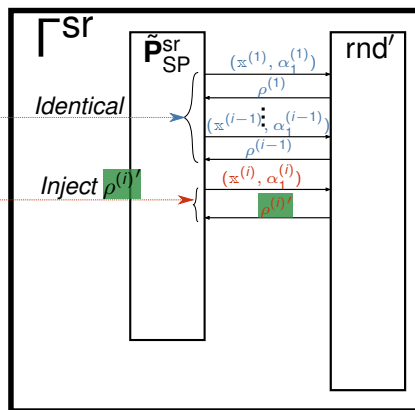
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



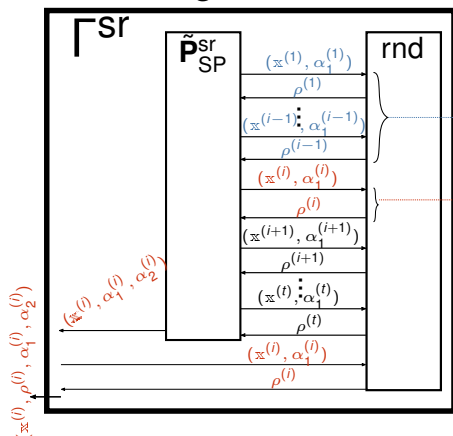
Needed



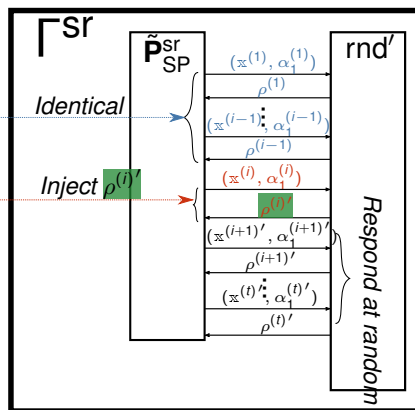
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



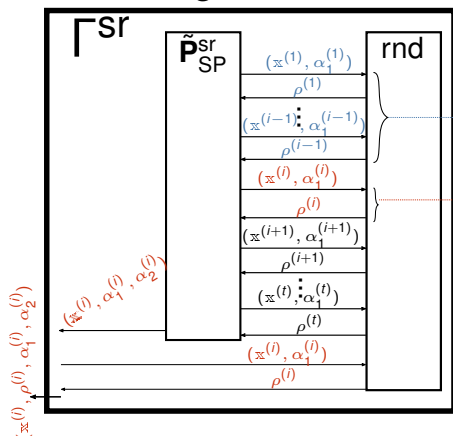
Needed



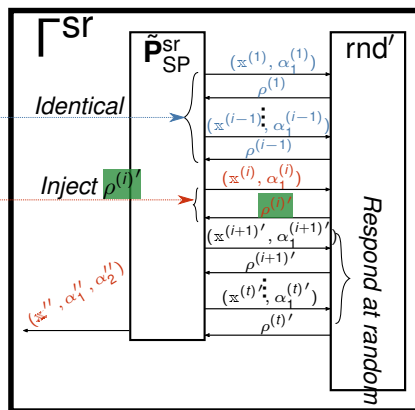
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



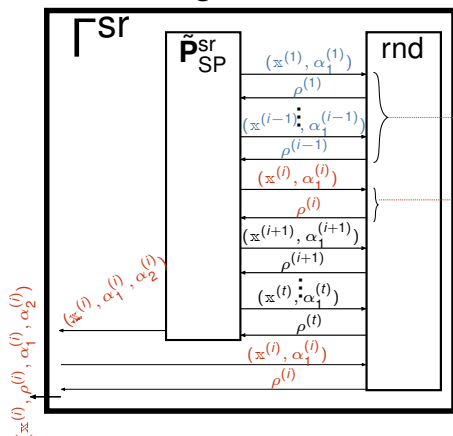
Needed



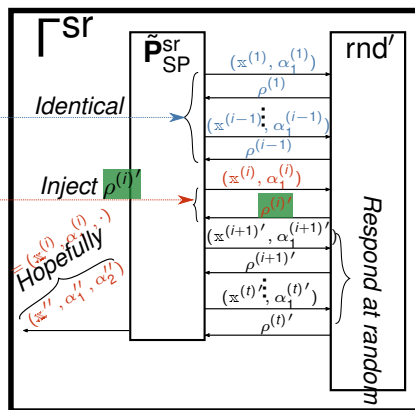
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



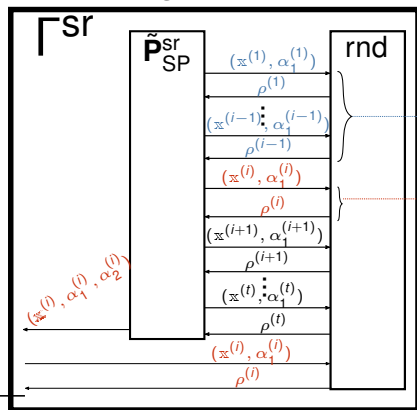
Needed



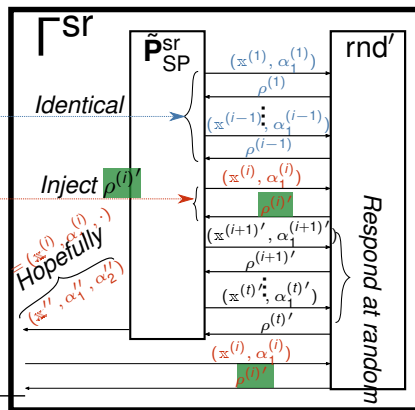
Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

Original run



Needed



Challenge substitution

Suppose the extractor is given a transcript of the form $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)}, \cdot)$ and we want to test $(\mathbb{x}^{(i)}, \alpha_1^{(i)}, \rho^{(i)'}, \cdot)$.

In other words, we must check that

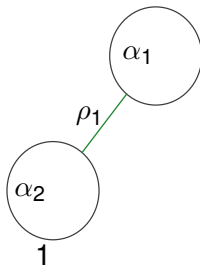
$$(\textcolor{red}{\mathbb{x}}^{(i)}, \textcolor{red}{\alpha}_1^{(i)}, \textcolor{blue}{\rho}^{(i)'}, \cdot) \leftarrow \Gamma^{\text{sr}}(\text{rnd}', \tilde{\mathbf{P}}_{\text{SP}}^{\text{sr}}) .$$

This is known as the *consistency check*.

The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

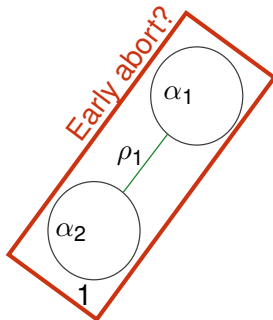


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;

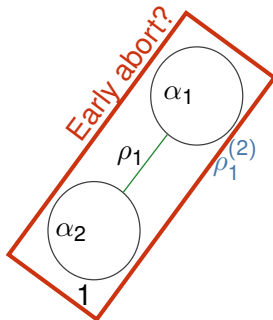


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;

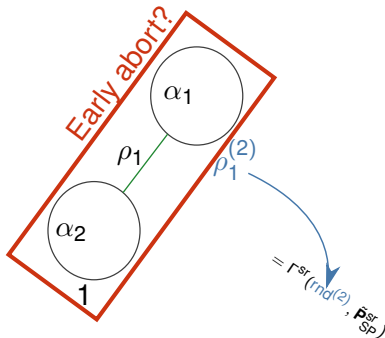


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that
 $\alpha_1 = \alpha_1'' \wedge \mathbb{x} = \mathbb{x}''$ where
 $(\mathbb{x}, \alpha_1'', \rho_1^{(2)}) \xleftarrow{\text{tr}^{\text{sr}}}$
 $\Gamma^{\text{sr}}(\text{rnd}^{(2)}, \tilde{\mathbf{p}}_{\text{IP}}^{\text{sr}})$;

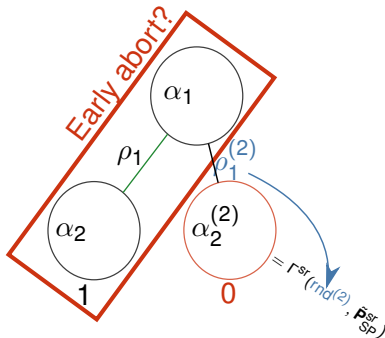


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that
 $\alpha_1 = \alpha_1'' \wedge \mathbb{x} = \mathbb{x}''$ where
 $(\mathbb{x}, \alpha_1'', \rho_1^{(2)}) \xleftarrow{\text{tr}^{\text{sr}}}$
 $\Gamma^{\text{sr}}(\text{rnd}^{(2)}, \tilde{\mathbf{p}}_{\text{IP}}^{\text{sr}})$;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ (OK) and the verifier accepts;

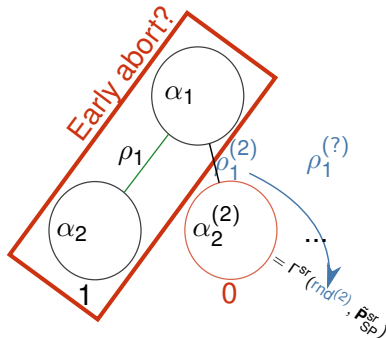


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that
 $\alpha_1 = \alpha_1'' \wedge \mathbb{x} = \mathbb{x}''$ where
 $(\mathbb{x}, \alpha_1'', \rho_1^{(2)}) \xleftarrow{\text{tr}^{\text{sr}}} \Gamma^{\text{sr}}(\text{rnd}^{(2)}, \tilde{\mathbf{p}}_{\text{IP}}^{\text{sr}})$;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ (OK) and the verifier accepts;
- 5 Continue until we have found enough good challenges:

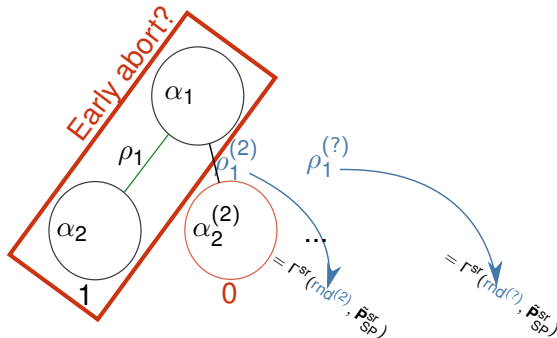


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that
 $\alpha_1 = \alpha_1'' \wedge \mathbb{x} = \mathbb{x}''$ where
 $(\mathbb{x}, \alpha_1'', \rho_1^{(2)}) \xleftarrow{\text{tr}^{\text{sr}}}$
 $\Gamma^{\text{sr}}(\text{rnd}^{(2)}, \tilde{\mathbf{p}}_{\text{IP}}^{\text{sr}})$;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ (OK) and the verifier accepts;
- 5 Continue until we have found enough good challenges:

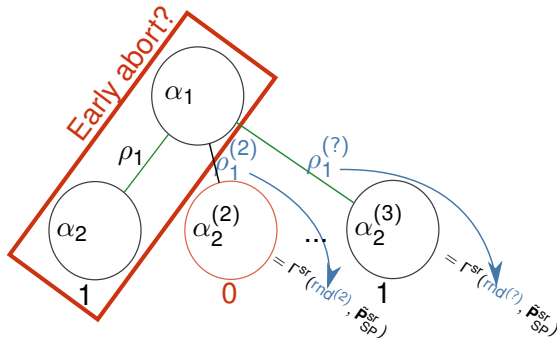


The complete extractor

Suppose we need a a -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.

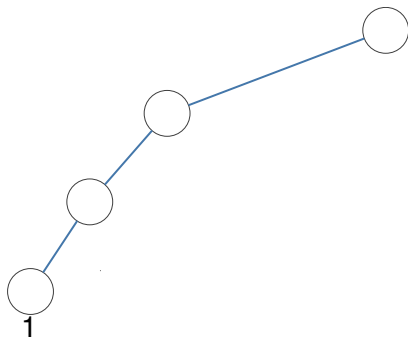
Extractor:

- 1 Early abort?;
- 2 Pick $\rho_1^{(2)} \leftarrow \mathcal{R} \setminus \{\rho_1\}$;
- 3 Create a random oracle $\text{rnd}^{(2)}$ such that
 $\alpha_1 = \alpha_1'' \wedge \mathbb{x} = \mathbb{x}''$ where
 $(\mathbb{x}, \alpha_1'', \rho_1^{(2)}) \xleftarrow{\text{tr}^{\text{sr}}} \Gamma^{\text{sr}}(\text{rnd}^{(2)}, \tilde{\mathbf{p}}_{\text{IP}}^{\text{sr}})$;
- 4 If the challenge returned by the game is $\rho_1^{(2)}$ (OK) and the verifier accepts;
- 5 Continue until we have found enough good challenges:



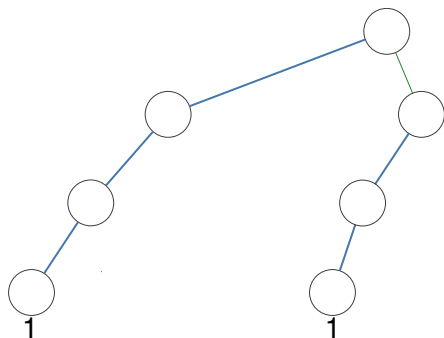
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for x and that the set of challenges is $\mathcal{R}$.



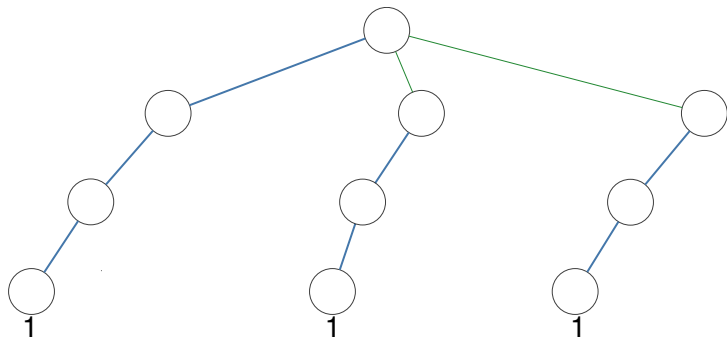
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



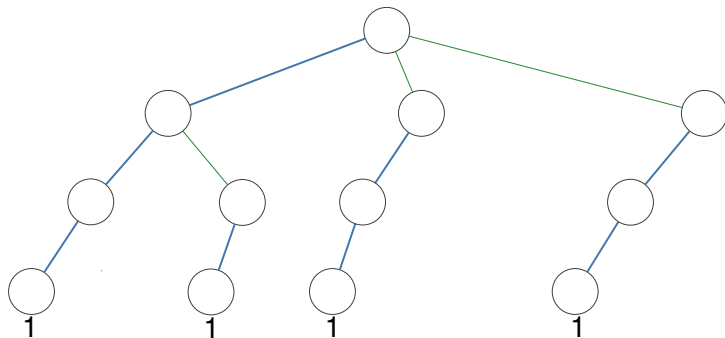
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.



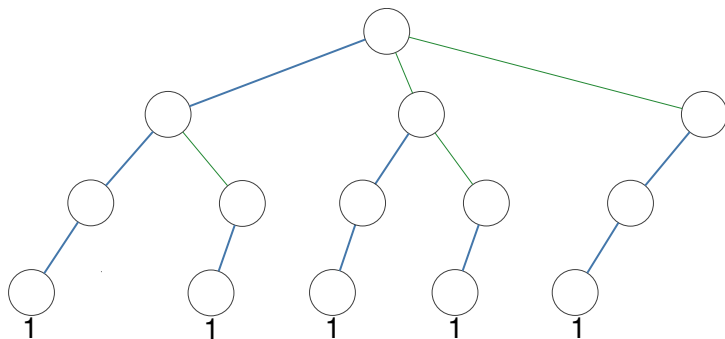
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



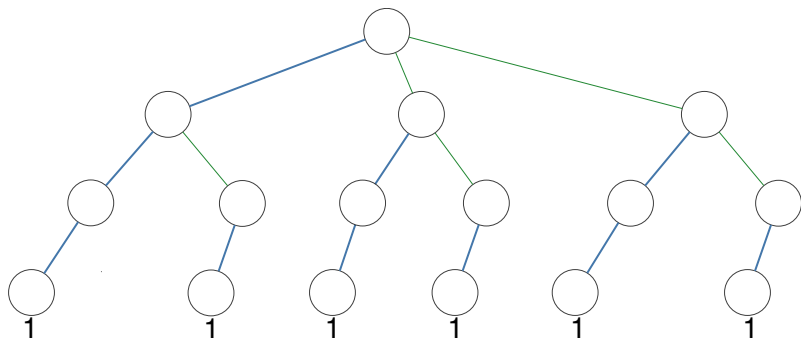
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.



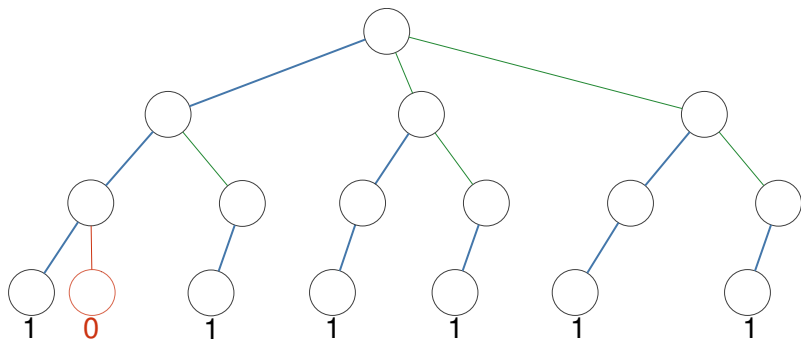
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



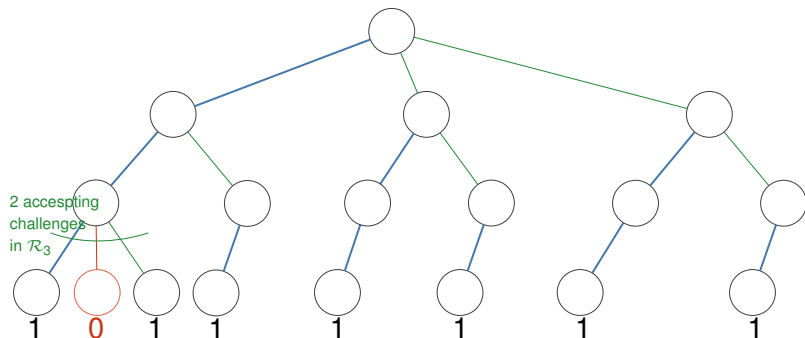
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



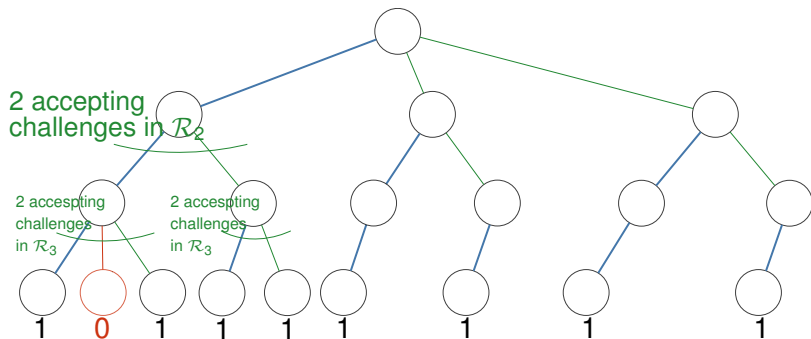
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



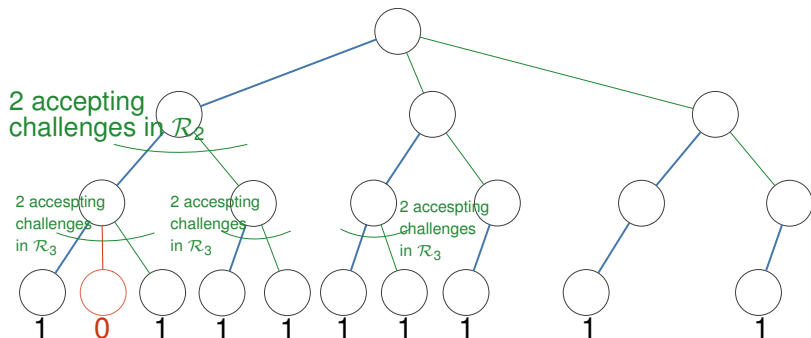
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



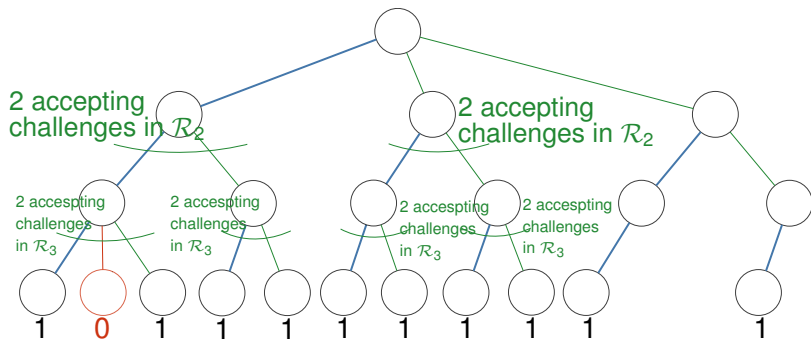
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



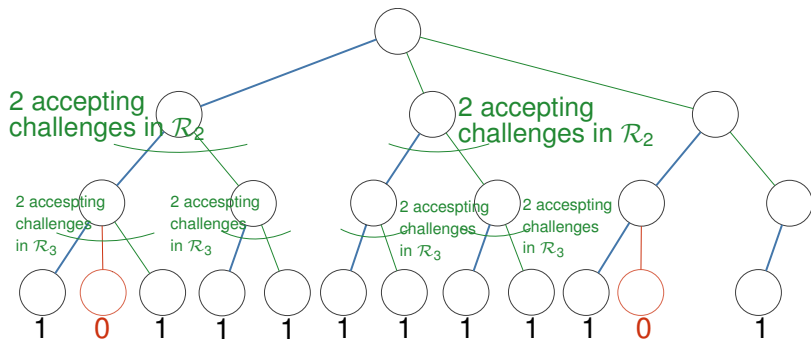
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.



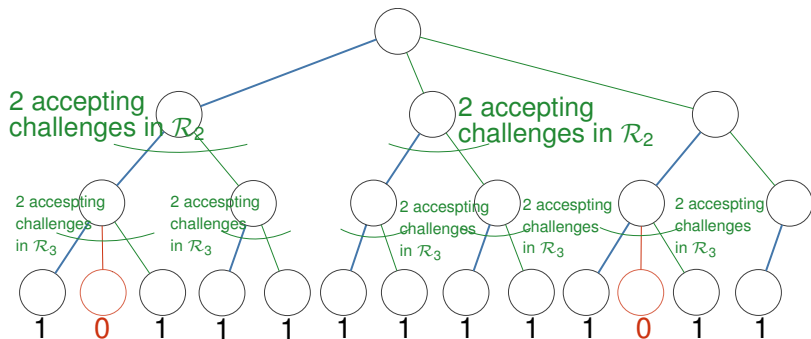
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{x}$ and that the set of challenges is $\mathcal{R}$.



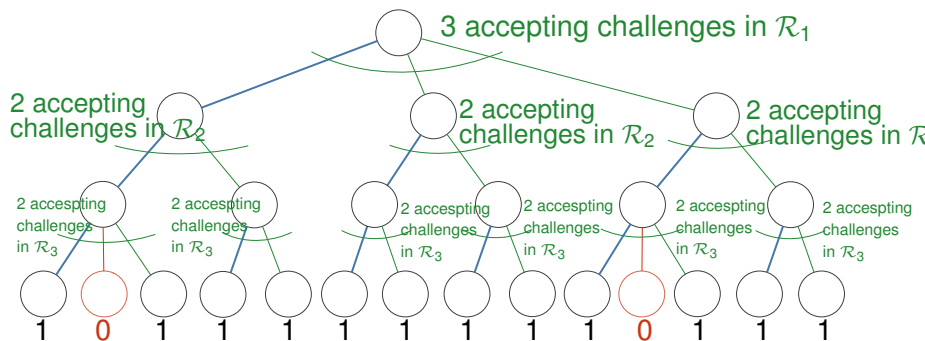
How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



How to generalize to IPs

Suppose we need a $(3, 2, 2)$ -tree of accepting transcripts for $\mathbb{X}$ and that the set of challenges is $\mathcal{R}$.



The result

Theorem

Let $\text{IP} = (\mathbf{P}_{\text{IP}}, \mathbf{V}_{\text{IP}})$ be a public-coin interactive proof for a relation R with $(a_1, \dots, a_k)$ -special soundness and verifier randomness complexity $(r_1, \dots, r_k)$. It follows that IP has rewinding state restoration knowledge soundness error

$$\kappa_{\text{IP}}^{\text{sr}}(\mathbb{X}, t) \leq (t + 1) \left(1 - \prod_{i=1}^k \left(1 - \frac{a_i - 1}{2^{r_i}} \right) \right) ,$$

and extraction time

$$\text{et}_{\text{IP}}^{\text{sr}}(n, t, \tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}) \leq \left(\left(\prod_{i=1}^k a_i \right) + (t + 1) \cdot \left(\left(\prod_{i=1}^k a_i \right) - 1 \right) \right) \cdot (\tau_{\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}} + \text{poly}(n)) .$$

Moreover, the number of times the knowledge soundness extractor invokes $\tilde{\mathbf{P}}_{\text{IP}}^{\text{sr}}$ is $\left(\left(\prod_{i=1}^k a_i \right) - 1 \right) (t + 1)$.

Further research

Limitations of special soundness

- Some protocols, such as *Brakedown*, are special sound with too large arity;

Limitations of special soundness

- Some protocols, such as *Brakedown*, are special sound with too large arity;
- **AttemaFR23** have introduced a weakening of special soundness called Γ -*special soundness*;

Limitations of special soundness

- Some protocols, such as *Brakedown*, are special sound with too large arity;
- **AttemaFR23** have introduced a weakening of special soundness called Γ -special soundness;
- Moreover, **AttemaFKR2023** have proved that the Fiat–Shamir transformation of a Γ -special sound protocol is knowledge sound.

Limitations of special soundness

- Some protocols, such as *Brakedown*, are special sound with too large arity;
- **AttemaFR23** have introduced a weakening of special soundness called Γ -*special soundness*;
- Moreover, **AttemaFKR2023** have proved that the Fiat–Shamir transformation of a Γ -special sound protocol is knowledge sound.

Therefore, it would be interesting to see if

Limitations of special soundness

- Some protocols, such as *Brakedown*, are special sound with too large arity;
- **AttemaFR23** have introduced a weakening of special soundness called Γ -special soundness;
- Moreover, **AttemaFKR2023** have proved that the Fiat–Shamir transformation of a Γ -special sound protocol is knowledge sound.

Therefore, it would be interesting to see if

- 1 *Brakedown* is Γ -special sound;

Limitations of special soundness

- Some protocols, such as *Brakedown*, are special sound with too large arity;
- **AttemaFR23** have introduced a weakening of special soundness called Γ -special soundness;
- Moreover, **AttemaFKR2023** have proved that the Fiat–Shamir transformation of a Γ -special sound protocol is knowledge sound.

Therefore, it would be interesting to see if

- 1 *Brakedown* is Γ -special sound;
- 2 Γ -special soundness implies state restoration knowledge soundness.

Questions

References I